

Lecture 10 — Conclusion & Worked Example (Drawlets)

Lecture id: L10 **Source decks:** Conclusion (~10p), Example of software change / Drawlets (~38p)

Labs: None **Process phase(s):** Conclusion (+ full change-process walkthrough) **Citation key:**

(Conclusion p.X) , (Drawlets p.X) ; readings [Raj13] (Rajlich, *Software Engineering: The Current Practice*, 2012 — Ch. 17 is the source of the Drawlets example), [Fowler99] (Fowler, *Refactoring*), [GHJV94] (Gang of Four, *Design Patterns*), [MC09] (Martin/Clean Code), [Martin] (Clean Architecture). **Grounding note:** Every non-obvious claim is cited to a specific source slide. The

Conclusion deck supplies the closing-phase theory (commit, baseline, release). The **Drawlets** deck is a single end-to-end worked change on a 40 000-LOC drawing framework and is the centerpiece of this guide; its class-diagram slides use a colour code (green = comprehended, orange = currently-examined/"on the fly", red = located concept / class needing change, blue boxes = new classes, blue arrows = propagation direction, grey = backtracked/dead-end) that the slide text cannot convey, so those visual states are reconstructed and described in prose here. Where a slide is title-only (a diagram), the page is still cited so you can find the figure. This is a retrieval guide; it summarises and organises the slides, it does not add outside facts beyond the named readings.

Overview

Lecture 10 closes the SB5-MAI change-process arc in two complementary halves.

Half 1 — Conclusion (the last phase). After a change has been actualized, postfactored, and verified, the work must be *concluded*: returned to the team and, eventually, to users. The Conclusion deck breaks this into three steps — **Commit** (return updated code to the configuration-management repository, resolving conflicts), **New baseline** (thorough team-wide testing that certifies the integrated code as the new agreed-upon version), and **New release** (delivering baseline code to actual users) (Conclusion p.1–2). It stresses that *the precise activities depend on the specific software process* (Conclusion p.1): a continuous-integration shop concludes differently from a quarterly-release shop, but the same three logical steps occur. The deck then drills into baseline mechanics: baseline testing guarantees progress-not-regression, takes significant time (often overnight/weekend, run by a specialised testing team), and acts as a **deadline** — miss it and you re-submit at the next baseline, with management tracking misses (Conclusion p.3–5). It distinguishes **bugs in baseline** (minor → still certify, push fixes onto the bug stack) from a **broken baseline** (serious → testing team rejects the commits, the whole increment can be invalidated, reputations suffer) (Conclusion p.6–7). Stakeholders close the loop with **acceptance testing** and approve software for release (Conclusion p.8). Releases come in two cadences — large downloadable releases and small **patches** merged into the user's program — under a "versioned model of software lifespan" (Conclusion p.9–10).

Half 2 — Worked example (Drawlets). The second deck walks *one change request* through the *entire* eight-phase process on the **Drawlets** drawing framework: add an **owner** to every figure so only the owner may modify it (Drawlets p.6). You watch concept location hunt for "figure properties," take a wrong path, **backtrack**, and find the right class; impact analysis estimate the blast radius; actualization implement two new classes (`OwnerIdentity`, `SimpleListener`) and propagate the change across ~13 classes step by step; verification run 385 unit tests + 141 functional tests; and a closing **refactoring** study show how *moving code* and *splitting roles* would shorten that propagation — quantified in a table comparing "no refactoring" vs. "move function" vs. "splitting" (Drawlets p.16–38). The punchline: a change that *touches* only ~91 production

lines (0.5 % of the code) nonetheless *ripples* through a large part of the class graph, and good structure (or refactoring) is what keeps that ripple short.

Read together: Half 1 tells you *how a change ends*; Half 2 shows you *every phase that came before the end*, made concrete.

How the two decks map onto the change mini-process

The Conclusion deck's opening slide shows the full phase diagram of the change mini-process — **Initiation** → **Concept Location** → **Impact Analysis** → **Prefactoring** → **Actualization** → **Postfactoring** → **Conclusion**, with **VERIFICATION** drawn as a vertical band spanning all of them — and highlights Conclusion as "the last phase of software change" (Conclusion p.1). Every slide of both decks can be placed onto exactly one of those phases (or onto the Verification band), which makes the lecture a complete index of the process:

Phase	Conclusion deck slides	Drawlets deck slides
(System context — what is being changed)	—	p.1–5 (Drawlets, host application, <code>SimpleApplet</code> window, top classes)
Initiation	—	p.6 (change request + rationale)
Concept Location	—	p.7–15 (concepts, classification, locating figure properties, wrong way ×2, backtrack ×2, right way, summary)
Impact Analysis	—	implicit in the class graph of p.5/p.9 (the located class's neighbourhood foreshadows the ripple)
Prefactoring	—	not performed (discussed retrospectively as the refactorings of p.31–36)
Actualization	—	p.16 (new classes), p.17–24 (change propagation start → done)
Postfactoring / Refactoring study	—	p.31–38 (shorten propagation: moving the code, splitting roles, numerical data, conclusions)
Conclusion	p.1–10 (the entire deck: commit, baseline, deadline, bugs/broken baseline, stakeholders, release, patches)	—
Verification (vertical band)	p.1 (the VERIFICATION band itself), p.3–4 (baseline testing), p.8 (acceptance testing)	p.25–30 (unit tests, functional tests, acceptance-test creation, actualization's test work, test-suite maintenance, results)

Two reading tips follow from this map. First, the Drawlets deck *never shows* an explicit Conclusion slide — the Conclusion deck *is* that phase for the worked example, which is why the two decks are taught as one lecture. Second, verification material appears in *both* decks (baseline/acceptance testing in Conclusion p.3–4/p.8; the test-suite numbers in Drawlets p.25–30), mirroring the diagram's claim that Verification spans every phase rather than being a phase of its own (Conclusion p.1).

Learning Objectives

After this lecture you should be able to:

1. **Name and order the conclusion's three steps** — commit, new baseline, new release — and explain that their concrete form depends on the team's software process (Conclusion p.1–2).
 2. **Explain baseline testing**: why it must be thorough (progress-not-regression), why it is expensive (overnight, specialised team), and how its frequency trades bug-accumulation against overhead (Conclusion p.3–4).
 3. **Treat the baseline as a deadline** and describe what happens when a programmer misses it (re-submit next baseline, extra integration work, management visibility) (Conclusion p.5).
 4. **Distinguish "bugs in baseline" from "broken baseline"** and the testing team's options in each case (certify-and-defer vs. reject-and-invalidate) (Conclusion p.6–7).
 5. **Place stakeholder acceptance testing and release cadence** (large releases vs. merged patches; versioned lifespan) in the conclusion (Conclusion p.8–10).
 6. **Walk a single change request through all eight phases** on Drawlets, naming what happens in each (Drawlets, whole deck).
 7. **Perform concept location as a graph search** with backtracking — recognise the wrong-way/right-way pattern and explain why dependency *direction* matters (Drawlets p.9–15).
 8. **Trace change propagation** as marking-and-revisiting: a changed class makes its neighbours inconsistent, you visit each, change-or-confirm, until the marks clear (Drawlets p.16–24).
 9. **Read the verification numbers**: 385 unit tests / 1369 assertions / 4800 LOC of test code, 141 functional tests, ~1.4 test LOC per modified production LOC (Drawlets p.25–30).
 10. **Evaluate refactoring as propagation-shortening** — "move code into fewer classes" and "split the roles" — and read the cost table comparing the three strategies (Drawlets p.31–38).
-

Key Concepts

Concept 1 — The Conclusion phase and its three steps

What it is. Conclusion is the **last phase** of the change mini-process — the wind-down that takes a change which has already been *coded, refactored and tested in isolation* and formally *folds it back into the shared codebase and out to the world*. It sits below Postfactoring in the phase diagram, still spanned by the vertical **Verification** band that wraps every phase, so testing does not stop just because you have reached the end (Conclusion p.1). Crucially, the deck is explicit that the phase has *no single recipe*: its activities *depend on the specific software process* (Conclusion p.1) — a continuous-integration shop and a quarterly-release shop perform the same three logical steps very differently.

What it's used for. This phase exists to answer the question "the change works on my machine — now what?" It is the hand-off machinery that turns a private edit into shared, certified, shipped software. Without it a finished change is worthless: it never reaches teammates, never gets integration-tested against everyone else's work, and never reaches users. The three steps each guarantee a different thing — integration, certification, and delivery.

The three logical steps (each a wider blast radius and a higher bar):

- **Commit.** Programmers return their updated code to the **configuration-management repository**, *resolving conflicts* with whatever others committed in the meantime (Conclusion p.2). This is the Git/CM step: your local change becomes part of the shared trunk. *What it's for*: it makes the change visible to the whole team and exposes it to merge conflicts so they are resolved now, by you, rather than later, by someone guessing. *Example*: you `git pull`, hit a conflict in `AbstractFigure` because a colleague also edited it, merge the two intents, and push.

- **New baseline.** The committed code is integrated and tested *as a whole*; if it passes, it becomes the team's new agreed reference version that everyone now builds on (Conclusion p.2–3). *What it's for:* commit only proves your file compiles in isolation; the baseline proves the *combined* system still works, certifying it as the trustworthy "known-good" point.
- **New release.** From time to time the baseline is delivered to *users* (Conclusion p.2, p.9). *What it's for:* the baseline is internal; the release is the only step that actually puts value in users' hands.

The mental model: *commit* hands code to the **team**, *baseline* certifies it for the **project**, *release* ships it to **users**. Each step is a wider audience, a wider blast radius, and a higher bar to clear.

Concept 2 — Baseline and baseline testing

What it is. A **baseline** is a tested, agreed-upon snapshot of the *whole* system — every committed change integrated together — that the team treats as the official "known-good" version and builds the next round of work on top of. **Baseline testing** is the act of certifying a candidate snapshot into that status: running **thorough testing** over the integrated whole until two guarantees hold — the new baseline is *as bug-free as possible*, and it represents *progress of the project, not a regression* (Conclusion p.3).

What it's used for. The baseline is the team's shared point of reference and synchronisation. Because everyone develops against the last baseline, it must be trustworthy: if it were buggy, every programmer would inherit those bugs into their own work. The decisive guarantee is "*not a regression*" — the new baseline must never be **worse** than the previous one on *already-working* functionality. That is exactly what a regression test suite checks: new value may be added, but old value must not be broken. The baseline is therefore the mechanism that lets a team move forward confidently instead of accumulating silent breakage.

When & how it's applied. Baseline testing is **expensive** and deliberately scheduled, not run on every commit: it can take significant time, is *often done overnight or over the weekend*, and is conducted by a *specialised testing team* rather than the developers themselves (Conclusion p.3) — so it is typically kicked off automatically at, say, 6 p.m. and its results reviewed the next morning. Its **frequency** is a genuine tuning decision driven by program size and required quality, with a cost on *both* sides (Conclusion p.4):

- Postpone too long → **bugs accumulate** between baselines, so when testing finally runs there are many faults tangled together, making diagnosis and the eventual testing task much harder (Conclusion p.4).
- Test too frequently → **unnecessary overhead**, because each run consumes machine time and the testing team's effort for little incremental gain (Conclusion p.4).

So a large, safety-critical program baselines often despite the cost; a small low-stakes one baselines rarely. The point of the trade-off is that "more testing is always better" is *false* once you count overhead.

Concept 3 — Baseline as a deadline; missed deadlines

What it is. Because baseline testing runs on a schedule, the *moment it starts* becomes a hard **deadline to commit**: any change not committed by then is simply not in this baseline (Conclusion p.5). The deadline is the cut-off line between "made it into this integration" and "wait until next time."

What it's used for. The deadline turns the baseline from a passive snapshot into an active *synchronisation point* that forces the team to converge their work at a known instant — and, simultaneously, into an *accountability mechanism*. Both roles matter: technically, it ensures everyone's code is present and consistent before the expensive test run; socially, it makes lateness visible and chargeable to an individual.

When & how it's applied. If a programmer **misses the deadline**, their work is not in this baseline; they must **submit by the next deadline** (Conclusion p.5). That is not free: because the baseline has *moved on* in

the meantime (other people's changes are now integrated), the latecomer must re-integrate their work against the new, different baseline — *additional work*, and *sometimes significant additional work*, because the code they branched from no longer exists in that form (Conclusion p.5). On top of the technical cost there is a managerial one: management *knows how often a particular programmer missed the deadline* and *may require an explanation* (Conclusion p.5). *Example*: you finish a feature an hour after the nightly baseline kicks off; tomorrow the trunk already contains three other people's merged work, so before you can submit you must rebase your change onto it, re-resolve conflicts, and re-test — and your name is on the "missed" list. The lesson: treat the deadline as **hard**, never soft.

Concept 4 — Bugs in baseline vs. broken baseline

What it is. When committed code is faulty, the bugs surface *during baseline testing* — the certification run is precisely where faulty commits are caught (Conclusion p.6). The two terms name the *two possible verdicts* the testing team can reach about those bugs, distinguished by **severity**:

- **Bugs in baseline (minor).** The faults are small enough that the system still essentially works. The testing team *can still certify the updated code as the new baseline*; the bug corrections are *added to the stack of bug reports* and *fixed as part of future changes* (Conclusion p.6). *What this is for / what it guarantees*: it lets progress continue rather than stalling the whole team over cosmetic or low-impact defects — value ships now, the debt is logged and paid down later. *Example*: a tooltip renders the wrong colour. Certify, file it, move on.
- **Broken baseline (serious).** The faults are bad enough that the integrated system is not trustworthy. The testing team *rejects the buggy commits* (Conclusion p.7). Sometimes *the whole work done on the new baseline has to be rejected* — **no new baseline is created**, all the increment's work is *invalidated or postponed*, which is *a significant cost on large projects* because everyone's contributions for that cycle are thrown back (Conclusion p.7). *What this is for*: it protects the team from inheriting a broken foundation — better to lose one increment than to build the next ten on rotten code. There is also an accountability edge: the team can *identify the programmers who committed buggy files*, and *their reputation suffers* (Conclusion p.7). *Example*: a commit makes the canvas crash on load — the entire baseline is rejected until it is removed or fixed.

Why it matters. The distinction is a *severity judgement* made by the testing team, and it directly drives both the **schedule** (defer-to-stack keeps the train moving; reject-and-invalidate derails it) and **individual accountability** (named programmers, damaged reputation). It is the single most-tested contrast in this deck.

Concept 5 — Stakeholder role and acceptance testing

What it is. **Acceptance testing** is *functional testing done by the stakeholders* — the customers, product owners, or end-user representatives, not the development or testing team — that *thoroughly tests software functionalities* against what was actually asked for (Conclusion p.8). It checks the software from the *outside*, in user terms ("can only the owner modify a figure?"), rather than in code terms.

What it's used for. It serves two purposes at once (Conclusion p.8). First, it **gives stakeholders information about the project's progress** — by exercising the real features, stakeholders see for themselves how far the project has come, rather than trusting a status report. Second, and decisively, it is the **gate by which stakeholders approve software for the release**: acceptance testing is the sign-off that converts a technically-certified baseline into a *business-approved* one. No acceptance, no release.

When & how it's applied. It comes *after* the internal certification and *before* delivery, so the order of trust is two-layered: the **testing team certifies the baseline (internal, "does it work technically?")**, then

stakeholders accept it (external, "is it what we wanted?"), and only then does it go out. *Example in this deck*: once the owner feature is in a baseline, stakeholders run through the user flows — log in with an ID, draw figures, confirm a *different* user cannot move yours — and, satisfied, approve the owner feature for release (Conclusion p.8). The Drawlets deck even supplies the machinery used: a record/playback tool plus JGiven and Mockito drive these functional checks (Drawlets p.27).

Concept 6 — New release and release cadence

What it is. A **release** is the act of delivering baseline code to *users* — the only step in the whole process that puts the change in the hands of the people it was built for (Conclusion p.9). It is distinct from a commit (to the team) or a baseline (to the project): a release crosses the final boundary, out of the organisation entirely.

What it's used for / why it matters. Releasing is not a trivial copy: it requires *substantial extra work* — packaging, installers, migration, documentation, support — and its *frequency is both a technical and a business decision* (Conclusion p.9). It is "technical" because each release costs engineering effort and risk; it is "business" because release timing affects customers, contracts, and revenue. That dual nature is why product managers, not just engineers, own the release calendar.

When & how it's applied. The common pattern is a *mix* of cadences: **less-frequent large releases + more-frequent small releases** (Conclusion p.9, e.g. "AwesomeApp 4.2"), a scheme the deck calls the *"versioned model of software lifespan"* (Conclusion p.9) — the major number jumps on big releases, the minor number ticks on small ones. The *delivery mechanism* differs by size (Conclusion p.10): **large releases** are *downloaded and installed* as a whole new build (the user replaces the application); **small releases** are delivered as **patches** that are *incorporated into the user's program by the tool "merge"* — i.e. only the changed pieces are merged into the already-installed software, so users get a fix without reinstalling everything. *Example*: the owner feature, if large, ships as "AwesomeApp 4.2" for download; a follow-up bug-fix ships as a merge-applied patch.

Concept 7 — Concept location as graph search with backtracking

What it is. Concept location is the activity of finding *where in the code* a concept from the change request actually lives — the class (or classes) you must edit to implement it. On Drawlets it is shown not as a lookup but as a **search over the class dependency graph** that can go **wrong** and must **backtrack** (Drawlets p.9–15): you start from a class you already understand, follow dependency edges, examine a candidate class "on the fly," and either *confirm* it holds the concept or *mark it a dead end and back up* to try another path.

What it's used for. Before you can change code you must know *which* code — and in an unfamiliar 40 000-LOC framework that is genuinely hard. Concept location is the disciplined way to find the right starting class without reading the whole system. Framing it as *search* (with the possibility of error) is the whole point: it sets the correct expectation that you will form a hypothesis, be wrong sometimes, and recover, rather than expecting to "just know" where things are.

When & how it's applied. The Drawlets slides make the search *visible* by colouring each class by its current search state — green = already comprehended, orange = currently-examined/"on the fly," red = the located target, grey = abandoned/dead-end (colours reconstructed from the figures, since the slide text cannot convey them). This turns concept location into a watchable walk through the graph: comprehended classes anchor the start, the orange frontier advances, dead branches go grey, and the target lights up red. The full step-by-step trace — including the instructive wrong turn toward the *container* `SimpleDrawingCanvas` and the correct turn toward the *abstraction* `AbstractFigure` — is in the Worked Example below.

Concept 8 — Change propagation as a marking process

What it is. Change propagation is what happens *after* you edit the located class. Changing one class makes it **inconsistent** with its neighbours (their assumptions about it no longer hold), so propagation is the systematic process of visiting each affected neighbour and either *changing it* to restore consistency or *confirming it still works* unchanged — and each class you change can in turn make *its* neighbours inconsistent, so the work spreads outward as a **wave through the dependency graph** until no inconsistencies remain (Drawlets p.16–24).

What it's used for / why it matters. Propagation is the mechanism that prevents a "fix" from silently breaking everything that depended on the old behaviour. By treating inconsistency as a *mark* that must be cleared, it guarantees you do not stop changing too early (leaving hidden breakage) or wander without a finish line. The "mark-and-clear" framing is also what makes the cost of a change *measurable*: the number of classes that get marked is the size of the propagation, which Concept 10 shows is the true cost driver.

When & how it's applied. The Drawlets deck animates propagation as marks spreading across the class diagram — red = needs attention / changed, orange = the current frontier being worked, green = checked-and-consistent (colour states reconstructed from the figures). You begin at the edited class, follow dependency edges marking neighbours inconsistent, resolve them one by one, and finish at the slide titled "**Propagation — done**" when every mark is cleared and the system is internally consistent again with owners enforced (Drawlets p.24). The full per-slide trace is in the Worked Example, Phase 5.

Concept 9 — Refactoring to shorten propagation

What it is. Refactoring here is *restructuring the code without changing its behaviour* specifically so that future change propagation is shorter. The deck's premise is that a **long change propagation is a problem** — the more classes a change ripples through, the more expensive and risky it is — so it would be *advantageous to shorten it*, and **refactoring can shorten the change propagation** (Drawlets p.31).

What it's used for. It reframes refactoring (Prefactoring before a change / Postfactoring after) in pure *cost* terms: better structure means a shorter ripple means cheaper, safer future changes. Refactoring does not make *this* change's lines fewer — it makes the *next* change touch fewer classes. That is why a long propagation should be read as a *signal* that the structure is poor and a refactoring opportunity exists.

When & how it's applied. Two concrete techniques are studied (Drawlets p.31, detailed in Phase 6 of the Worked Example):

- **(a) Move code into fewer classes** — gather the code affected by the change into a single base class (here, extract the duplicated move logic into `moveFigure(...)` and pull it up into `ConstructionTool`) so the change touches one shared place instead of many copies.
- **(b) Split the roles** of an overloaded method — when one method does two jobs but only one job needs the change, split it into two methods (here `move(...)` vs. `secureMove(...)`) so only the role that needs the owner-check is updated and the other stays untouched.

The pay-off is quantified on Drawlets in Concept 10: roughly *halving* the classes touched while the lines edited barely move.

Concept 10 — The cost of a change is propagation, not edit size

What it is. This is the headline conclusion of the whole lecture, stated as a measured fact: the *true* cost of a change is the *size of the propagation* it triggers — how many classes you must visit and reconcile — not the *number of lines* you ultimately type.

What it's used for / why it matters. It corrects the most common intuition about software change cost. A naïve estimate counts edited lines and concludes "tiny change, tiny cost." The Drawlets numbers demolish that: the *owner* change **modified only 91 LOC of production code (0.5 % of the 17 800-LOC production baseline)** and **124 LOC of test code (2.5 % of the 4 800-LOC unit-test baseline)** — about **1.4 test-code lines per modified production line** (Drawlets p.30) — yet that 0.5 % edit **propagated through ~13 modified classes plus 2 new classes** (Drawlets p.16–24, p.37). Notably the *test* code changed *more* in percentage terms than the production code, so much of a change's real weight lands in verification.

When & how it's applied. Use this when *estimating* or *justifying* effort: count the classes a change is expected to ripple through (impact analysis), not the lines, and read a long propagation as a reason to refactor (Concept 9) before the *next* change. The decisive Drawlets table (Concept 10's evidence, Drawlets p.37) shows refactoring barely moving the line count (91 → 95 → 87) while roughly halving classes touched (13 → 8 → 5) — direct proof that propagation, not edit size, is the cost lever, and exactly why structure and refactoring matter.

Concept 11 — The complete concept-classification table (all 13 concepts)

What it is. The classification slide is a 13-row table that sorts *every* concept extracted from the change request into exactly one of three columns — **irrelevant**, **external**, **significant** (Drawlets p.8). The concepts in row order, with their verified column marks, are:

Concept	irrelevant	external	significant
implement	x		
owner		x	
figure			x
user	x		
canvas			x
allowed	x		
modify	x		
beginning	x		
session			
input	x		
ID		x	
password		x	
created	x		

(Drawlets p.8 — **figure** and **canvas** are printed in bold on the slide; the **session** row carries no mark in any column on the slide as rendered, an apparent omission worth knowing if an exam asks you to reproduce the table.)

How to read it. Eight concepts are **irrelevant** (implement, user, allowed, modify, beginning, input, created — plus the unmarked **session** row, which by elimination behaves as irrelevant in the rest of the deck): they are ordinary request-language words that do not correspond to any present or future code unit. Three concepts are **external** (owner, ID, password): they cross the system boundary and *do not yet exist* in the code — they are exactly the things the change will *introduce* (**OwnerIdentity** carries them, Drawlets p.16). Two

concepts are **significant** (figure, canvas): they are already implemented in the system's design, so they are the only valid *anchors* for the code search.

The decision rule the table encodes. You can only *locate* what already exists, so the search must start from a **significant** concept; the **external** concepts tell you *what to attach* once the location is found; the **irrelevant** ones are noise to be filtered before any code is opened. On Drawlets, "figure" wins over "canvas" as the primary anchor because the request says the owner is a property *of each figure* — the canvas merely hosts figures — and indeed the search question becomes "where are *figure properties*?" (Drawlets p.9, "Locating figure properties").

Concept 12 — The diagram colour code, decoded slide-state by slide-state

What it is. Every class-diagram slide in the Drawlets deck (p.5, 9–24, 32, 34) communicates the algorithm's *state* purely through fill colours and thick coloured arrows. The recurring states, verified against the rendered figures, are:

- **Green box/circle** — a class or interface that is *marked*: either already comprehended (the starting set on p.5: `StylePalette`, `SimpleApplet`, `ToolBar`, `ToolPalette`) or newly *scheduled for inspection* by the spreading change (e.g. `SimpleApplet` and `CanvasTool` on p.18; `LocatorConnectionHandle`, `StylePalette`, `SelectionTool`, `ConstructionTool`, `LabelTool` on p.21; the four leaf creation tools on p.23).
- **Orange box/circle** — the *currently examined* element, the frontier where the engineer is right now (`SimpleApplet` while being searched on p.10–12; the `DrawingCanvas` interface circle through most of the propagation; `ShapeTool` on p.23–24).
- **Red box/circle** — a *changed* element (during propagation) or the *located concept* (in concept location: `AbstractFigure` turns red on the Summary slide p.15). By the final slide, red marks the complete modified set (13 classes + the `Figure` interface, p.24).
- **Grey box/circle** — *inspected and dismissed*: in concept location a dead-end candidate that was backtracked away from (`ToolPalette` on p.11, `SimpleDrawingCanvas` on p.14); in propagation a class that was visited and confirmed to need *no change* (`ToolBar`, `ToolPalette`, `CanvasTool`, `Locator`, `SequenceOfFigures` on p.24).
- **Pink/violet box** — a transient "changed, currently settled" rendering of `AbstractFigure` on p.21, between its red appearances on p.16–20 and p.22–24; treat it as a visual variant of "already changed" rather than a distinct algorithmic state.
- **Light-blue box** — a *new* class added by the change (`OwnerIdentity`, `SimpleListener`, from p.16 onward).
- **Thick blue arrow** — the *correct/forward* search or propagation direction (the move to `DrawingCanvas` on p.12; the path summary on p.15; `AbstractFigure` → `Figure` on p.17–20; the fan-out from the red `Figure` interface on p.21; the marking of `ShapeTool`'s subclasses on p.23).
- **Thick violet/light-purple arrow** — a *wrong-way* search move about to be backtracked (down into the palette branch on p.10; down into `SimpleDrawingCanvas` on p.13).
- **Thick orange arrow** — inconsistency flowing out of a changed class during propagation (out of `SimpleDrawingCanvas` toward `DrawingCanvas` / `SimpleApplet` / `CanvasTool` / `SequenceOfFigures` on p.17–18; out of `SimpleApplet` toward the palettes on p.19).
- **Yellow / cyan boxes (p.34 only)** — the refactoring-impact slide uses its own pair: **yellow** for `ConstructionTool` (the class that *receives* the pulled-up `moveFigure(...)`) and **cyan** for the four creation tools (`RectangleTool`, `EllipseTool`, `RectangularCreationTool`, `PG_RectImageTool`) whose duplicated logic was consolidated.

Why it matters. Exam questions about "what does the colouring on slide X mean" reduce to this legend; more importantly, the legend is the algorithm: green = marked, orange = in hand, red = changed/located, grey = cleared, blue = new. Concept location and change propagation are both worklist algorithms, and the colours are the worklist made visible.

Concept 13 — Interfaces vs classes in the Drawlets graph (and the "1 interface modified")

What it is. The top-classes diagram uses UML *lollipop* notation: interfaces are drawn as small circles, classes as boxes (Drawlets p.5). Four named elements in the diagram are **interfaces**: `Locator`, `Figure`, `SequenceOfFigures`, and `DrawingCanvas`. Everything else in the diagram is a class. Drawlets as a whole has **35 interfaces** against more than 100 classes (Drawlets p.2), so the diagram shows only the top of each hierarchy.

Why it matters for the numbers. The numerical-data table reports, for every strategy, "**Interfaces modified: 1**" (Drawlets p.37). That one interface is `Figure`: on the propagation slides it is the *only* circle that turns red (from p.21 through the final p.24), because adding owner-aware behaviour to figures required extending the `Figure` interface itself, which is exactly what re-marked *all* of `Figure`'s clients (`LocatorConnectionHandle`, `StylePalette`, `SelectionTool`, `ConstructionTool`, `LabelTool`, plus the `Locator` and `SequenceOfFigures` interfaces) for inspection on p.21. The other three interfaces end the change grey (`Locator`, `SequenceOfFigures` — inspected, unchanged) or orange (`DrawingCanvas` — repeatedly visited as the hub between the canvas implementation and its clients, but not counted as modified).

The structural lesson. Changing a *class* (`SimpleDrawingCanvas`, `AbstractFigure`) marks its immediate dependents; changing an *interface* (`Figure`) marks *every implementor and every client at once* — which is why the single interface modification on p.21 produces the largest one-step fan-out in the whole propagation. Interfaces are high-leverage, high-blast-radius edit points; that is the graph-theoretic reason the propagation, having started at two classes, suddenly engulfs the whole tool tree the moment `Figure` goes red.

JHotDraw Connection

The course's running case study is **JHotDraw**; Lecture 10's worked example is its sibling, **Drawlets** — another small object-oriented **drawing framework** (lines, freehand, rectangles, rounded rectangles, triangles, pentagons, polygons, ellipses, text boxes, images) (Drawlets p.1). The parallels make the example transfer directly to JHotDraw:

- **Same domain, same vocabulary.** Both centre on a `Figure` abstraction with concrete shape subclasses and a **drawing canvas**, manipulated by **tools** from a palette (Drawlets p.5 shows `Figure`, `AbstractFigure`, `DrawingCanvas`, `SimpleDrawingCanvas`, `ShapeTool`, `RectangleTool`, `EllipseTool`, `SelectionTool`, `ConstructionTool`, `ToolPalette`, etc.). Anyone who has done a JHotDraw "add a new figure" or "add a handle" change will recognise this class graph immediately.
- **Framework with a host application.** Drawlets is an *application framework* that *adds a graphical display to a host application*; the host supplies the canvas, toolbars, and tool buttons, and the bundled host is `SimpleApplet`, which runs in any browser (Drawlets p.1, p.3). This mirrors JHotDraw's application/framework split.
- **Designed with patterns.** Drawlets is *more than 100 classes, 35 interfaces, ~40 000 LOC*, and marketed as a "*perfect API*" (Drawlets p.2). It is the worked-example drawing framework of Rajlich Ch. 17, standing in the lineage of the pattern-rich drawing frameworks that began with **HotDraw** (the Smalltalk original by **Kent Beck and Ward Cunningham**) and JHotDraw. Like JHotDraw it is a textbook of **design**

patterns [GHJV94] (Composite figures, Strategy tools, Observer/listeners — note the new `SimpleListener` class added by the change, Drawlets p.16), which is precisely why both are used to teach maintenance: well-patterned code makes propagation paths legible.

- **Why two case studies.** JHotDraw teaches the *phases in isolation* across the course; the Drawlets change teaches the *phases in sequence on one request*. The Drawlets owner-change is the L10 stand-in for "do a complete JHotDraw change end-to-end."

If an exam question says "trace a change through JHotDraw," the Drawlets walkthrough below is the template you reproduce: same `Figure` / canvas / tool graph, same concept-location-then-propagation rhythm.

The SimpleApplet window in detail (Drawlets p.4)

The screenshot slide shows the running host application and is worth being able to describe, because the *ID button* on it is the visible hook for the owner change (Drawlets p.4, p.6):

- **Top toolbar:** a row of icon buttons (clipboard-style edit operations and display controls) ending with an **"ID" button** — the control through which "users input their ID and password" at the beginning of a session per the change request (Drawlets p.4, p.6).
- **Left tool palette:** a vertical strip of drawing-tool buttons (selection arrow, text/label tool "A", line, freehand, and the shape tools), i.e. the on-screen face of the `ToolPalette` / `CanvasTool` hierarchy from the class diagram (Drawlets p.4–5).
- **Centre canvas:** the drawing surface showing one of each figure family the framework supports — a freehand line, a straight line, a text label, a polygon, a rectangle, a triangle, an ellipse, an embedded **Image** box, a rounded rectangle, and a pentagon — matching the figure list on the title slide (lines, free-hand lines, rectangles, rounded rectangles, triangles, pentagons, polygons, ellipses, text boxes, images) (Drawlets p.1, p.4).
- **Right style palette:** a column of colour swatches plus a **NO** (no-colour) option and buttons **Fill, Line, Text, Back, Fore, Apply** — the on-screen face of the `StylePalette` class (Drawlets p.4–5).

The window therefore visually maps one-to-one onto the four green "comprehended" classes of the top-classes diagram — `SimpleApplet` (the window itself), `ToolBar` (top), `ToolPalette` (left), `StylePalette` (right) — which is exactly *why* those four are the comprehension starting set for concept location: they are the parts of the system a user (and hence a new maintainer) can see and already understands (Drawlets p.4–5, p.9).

Worked Example / Process Walkthrough

This is the centerpiece. We trace **one** change request through **all eight phases** on Drawlets. Phase headings map to the SB5-MAI process: Initiation → Concept Location → Impact Analysis → Prefactoring → Actualization → Postfactoring → Conclusion → Verification.

The system under change

Drawlets: an application framework giving a host app a **drawing canvas** with many figure types; **>100 classes, 35 interfaces, ~40 000 LOC**, by Beck & Cunningham, ported to Java, a self-described "*perfect API*" (Drawlets p.1–2). The host application `SimpleApplet` (part of the library, runs in any browser) provides the canvas, toolbars and buttons (Drawlets p.3); its window shows the palette down the left, the canvas in the middle with drawn shapes, and style controls (Fill/Line/Text/Back/Fore/Apply, an **ID** button) down the right (Drawlets p.4). The **top classes** form the graph we will navigate: `SimpleApplet` → `StylePalette`/`ToolBar`/`ToolPalette`, the `Figure` / `AbstractFigure` hierarchy, `SequenceOfFigures`,

DrawingCanvas / SimpleDrawingCanvas, and the tool tree CanvasTool → SelectionTool/ConstructionTool → ShapeTool → {RectangleTool, EllipseTool, RectangularCreationTool, PG_RectImageTool}, plus PrototypeConstructionTool, LabelTool, Locator, LocatorConnectionHandle (Drawlets p.5). On slide 5 the four classes reachable straight from the host — StylePalette, SimpleApplet, ToolBar, ToolPalette — are highlighted, anchoring where comprehension starts.

Phase 1 — Initiation (the change request)

What this phase is. Initiation is where a change *begins to exist as work*: a request is written down and judged worth doing. Nothing is coded yet — the deliverable is a stated, justified request.

Why the phase exists. Every change must trace back to a concrete, agreed reason; Initiation is the gate that prevents work starting on a vague or unjustified idea. It pins down *what* is wanted and *why it is worth the cost* before anyone spends effort.

What was done here. The **change request**: *Implement an owner for each figure. An owner is the user who put the figure onto the canvas, and only the owner should be allowed to modify it. At the beginning of a session, users input their ID and password and become the owners of all figures created during that session.* (Drawlets p.6). The **rationale** — the business case that justifies initiating the change — is that it *makes SimpleApplet more versatile and useful* and adds *support for cooperative work* (Drawlets p.6): multiple users can draw on one canvas without overwriting each other's figures. So Initiation here produces both the *what* (owner-per-figure, identity at session start, owner-only modification) and the *why* (cooperative work), and deems the request worth doing.

Phase 2 — Concept Location (find where "figure" lives)

What this phase is. Concept Location takes the words of the request and finds the *one class in the code* where the change must start. **Why the phase exists:** in a >100-class framework you cannot edit what you cannot find; this phase converts a natural-language request into a precise code location, and does so methodically (extract concepts → classify them → search the graph) rather than by guesswork.

Extract concepts. Re-read the request and pull out the nouns/verbs that might map to code: *implement, owner, figure, user, canvas, allowed, modify, beginning, session, input, ID, password, created* (Drawlets p.7). *Why:* you cannot search for "the change" — you search for individual concepts, so first you list them.

Classify concepts against the existing system into **irrelevant** / **external** / **significant** (Drawlets p.8 table). *Why this step exists:* not every word is a code concept, and searching all of them would waste effort — classification filters the list down to the ones worth chasing: - **Significant** (already in the design, central to the change): **figure, canvas**. These are exactly the concepts to anchor the search on, because they already correspond to classes. - **External** (cross the system boundary, come from outside the system): **owner, ID, password**. These are *new* things the change introduces from outside, so they are *not* where you start searching — you cannot locate a concept that does not exist in the code yet. - **Irrelevant** (generic English, not a code concept here): *implement, user, allowed, modify, beginning, input, created*. Discarded so they do not distract the search.

What this gives us / why it matters. The classification tells us *where to search*: the **significant** concept **figure** is the anchor — find the class that owns figure *properties*, because that is where the new **owner** property (an *external* concept) must attach. Anchoring on **figure** rather than **owner** is the whole reason classification is done first.

Locate "figure properties" — and go the wrong way first. The deck shows concept location as a guided walk over the class graph, with each visited class shaded:

1. **Start at SimpleApplet** (the host, already comprehended) and ask: where are a figure's *properties* set/stored? (Drawlets p.9 — SimpleApplet highlighted as the comprehension start.)
2. **Wrong way (attempt 1):** follow SimpleApplet's dependency *downward* toward the canvas concept and land on SimpleDrawingCanvas, hypothesising figure properties live there (Drawlets p.10 "Wrong way" → p.13 shows the candidate SimpleDrawingCanvas examined and rejected). The canvas *holds* figures but is not where a single figure's *own* properties (like its owner) belong.
3. **Backtrack:** mark the dead-end branch abandoned and return to the last good class (Drawlets p.11, p.14 "Backtrack" — the abandoned classes go grey, e.g. ToolPalette / SimpleDrawingCanvas greyed out).
4. **Right way:** instead of going to the *container*, go to the **figure abstraction itself**. Follow the dependency from SimpleApplet / DrawingCanvas *up* to the **Figure** interface, then to its implementation **AbstractFigure** — the class where per-figure state lives (Drawlets p.12 "Right way", p.15 "Summary").
5. **Located concept:** the **Summary** slide marks **AbstractFigure** in red as the class that holds figure properties and therefore must carry the new **owner** (Drawlets p.15). AbstractFigure is the concept-location result.

The pedagogy of the wrong-way/backtrack/right-way sequence: concept location is **search, not lookup** — you form a hypothesis, test it against the code, and backtrack when the dependency direction (container vs. abstraction) proves your hypothesis wrong. *Why the wrong turn is instructive:* the natural first guess is the *container* that holds figures (SimpleDrawingCanvas), but a single figure's *own* property like its owner belongs on the *figure abstraction* itself, not on whatever collection happens to store it — so the lesson is that **dependency direction decides correctness** (go *up* to the abstraction, not *down* to the container), and that being wrong-then-backtracking is normal, not a failure.

Phase 3 — Impact Analysis (estimate the blast radius)

What this phase is. With AbstractFigure fixed as the change's origin, Impact Analysis *estimates* — before any code is written — which other classes the change will reach by following the dependency graph outward from that origin.

Why the phase exists. It sizes the job. Concept Location tells you *where to start*; Impact Analysis tells you *how big this will be*, so the team can plan, schedule, and decide whether to refactor first. It is the phase that distinguishes a one-class edit from a cross-cutting change *in advance*, when that information is still useful for estimation.

What was done here. The class graph already foreshadows the answer: many classes **create, hold, or move** figures, so adding an owner-check to figure modification will ripple to every tool that modifies figures and every container that stores them. Impact analysis predicts the propagation will reach the **tool tree** (ConstructionTool , ShapeTool and its subclasses, SelectionTool , PrototypeConstructionTool , LabelTool), the **canvas/sequence** (DrawingCanvas , SimpleDrawingCanvas , SequenceOfFigures), the **palettes** (StylePalette , ToolPalette), and SimpleApplet itself — i.e. roughly the whole upper graph.

How it's validated. The estimate is confirmed *retrospectively* by Actualization: ~13 classes modified, 2 added (Drawlets p.37). That the prediction (whole upper graph) matches the outcome (13 classes) is exactly the value of the phase — it told the team *before* coding that this is a non-trivial, cross-cutting change, not a quick fix, so they could budget for it.

Phase 4 — Prefactoring (optional clean-up before the change)

What this phase is. Prefactoring is *optional* refactoring done *before* actualization, to clean up the code so the upcoming change is easier and the propagation shorter. **Why the phase exists:** if Impact Analysis reveals a long, messy ripple, it is often cheaper to restructure first (move duplicated code together, split overloaded methods) and *then* make the change against the improved structure — paying a small restructuring cost to avoid a large propagation cost.

What was done here — and why it's still informative that "nothing" was done. The deck does *not* perform a separate prefactoring step for this change; it goes straight to actualization, and the refactoring discussion is presented *after* the change as an analysis of how it *could have been* shortened (Drawlets p.31–38). The takeaway for the phase: prefactoring was *deemed unnecessary* here, which is a legitimate outcome for an optional phase. But the closing refactoring study (Postfactoring) shows *exactly the prefactoring one would have done* to make this and future owner-related changes cheaper — move the duplicated move-logic into `moveFigure`, and split `move` / `secureMove` — and the numerical table proves it would have cut classes touched from 13 to 5. So the value of this phase, even when skipped, is the reminder that doing that clean-up *first* would have shortened the very propagation you are about to walk through. See Phase 6.

Phase 5 — Actualization (implement + propagate)

What this phase is. Actualization is where the change is *actually made* — new code is written and the edit is *propagated* across the dependency graph until the system is consistent again. **Why the phase exists:** it is the only phase that changes behaviour; everything before it (locate, estimate, prefactor) was preparation, and everything after (postfactor, conclude, verify) reacts to what is done here. It has two parts: introduce the new design elements, then propagate.

New design elements. Two **new classes** appear in the graph (drawn as blue boxes on the actualization slide): `OwnerIdentity` — *what it is:* the value object holding the user identity an owner carries (built from the ID/password entered at session start); *what it's for:* it gives the system a first-class representation of "who owns this," which is the external concept the request introduced — and `SimpleListener` — *what it is:* an Observer/listener (Observer pattern, [GHJV94]); *what it's for:* it wires the identity into the figure/canvas machinery so ownership is established and notified without hard-coupling the classes. After this, `SimpleDrawingCanvas` / `AbstractFigure` depend on the new classes (Drawlets p.16). The change *begins* at the located class: `AbstractFigure` (red) gains the owner concept, and `OwnerIdentity` / `SimpleListener` (blue) are introduced to support it (Drawlets p.16). *Why these two and not more:* the design adds the *minimum* new structure needed to carry and wire identity, leaving the rest to propagation.

Change propagation, step by step (Drawlets p.17–24). Read the colours as: **red** = class changed / now inconsistent and needing attention, **orange** = the current frontier node being worked, **green** = visited-and-made-consistent, **grey** = touched-but-not-needing-change, **blue arrows** = direction the inconsistency is propagating, **blue boxes** = the new classes:

- **Change propagation (start), p.17.** The edit at `AbstractFigure` (red) makes the central `Figure` node inconsistent (it becomes the active/green frontier marker), and the new `OwnerIdentity` / `SimpleListener` (blue) are now wired toward `SimpleDrawingCanvas` (red). The wave starts at the figure abstraction and points outward.
- **Propagation – 1, p.18.** The frontier moves from `Figure` / `SequenceOfFigures` down to `SimpleApplet` and `CanvasTool` (both turn green = examined and reconciled this step); `AbstractFigure` / `SimpleDrawingCanvas` stay red (still in play). Orange arrows show the inconsistency flowing `SimpleDrawingCanvas` → `SimpleApplet/CanvasTool`.

- **Propagation – 2, p.19.** Now `SimpleApplet` itself turns **red** (it *does* need changing — it manages sessions/IDs), and the wave reaches the **palettes**: `StylePalette`, `ToolBar`, `ToolPalette` go green (checked, no change needed) while orange arrows fan out from `SimpleApplet`. `CanvasTool` greys (touched, fine).
- **Propagation – 3, p.20.** The palettes settle to **grey** (confirmed unaffected); `SimpleApplet` and `SimpleDrawingCanvas` remain red; the frontier (orange) sits on `DrawingCanvas`. The wave is now consolidating which classes truly change vs. merely-checked.
- **Propagation – 4, p.21.** The wave reaches the **Locator / LocatorConnectionHandle** region and the **tool subtree**: `LocatorConnectionHandle`, `StylePalette`, `SelectionTool`, `ConstructionTool`, `LabelTool` light up green/active as the frontier (big blue arrows) drives toward `ConstructionTool` and `LabelTool`. `AbstractFigure` flips to **pink/violet** (reconciled-but-noted). `Figure` is now red (central inconsistency being resolved).
- **Propagation – 5, p.22.** Most of the upper graph is now **red** (`LocatorConnectionHandle`, `StylePalette`, `SimpleApplet`, `SelectionTool`, `ConstructionTool`, `PrototypeConstructionTool`, `AbstractFigure`, `Figure`): these are the classes that genuinely require edits. The frontier (blue arrows) pushes down into `ShapeTool` and `LabelTool` (green = just reconciled), heading for the concrete figure-tool leaves.
- **Propagation – 6, p.23.** The frontier reaches the **leaf figure tools** — `RectangleTool`, `EllipseTool`, `RectangularCreationTool`, `PG_RectImageTool` turn green as each creation tool is checked (blue arrows along the `ShapeTool` subtree); `ShapeTool` is the orange frontier. Almost everything above is red (changed) or grey (checked).
- **Propagation – done, p.24.** No inconsistencies remain. Final tally visible in the diagram: a large red set of **modified** classes spanning the figure hierarchy, palettes, canvas, and tool tree; grey classes that were checked but unchanged (`ToolBar`, `ToolPalette`, `CanvasTool`, `Locator`, `SequenceOfFigures`); and the two blue **new** classes `OwnerIdentity` / `SimpleListener`. Propagation is complete — the system is internally consistent again with owners enforced.

This is the heart of the lecture: **one edit at `AbstractFigure` propagated across roughly the whole upper graph**, visited class-by-class, each either changed (red) or confirmed (grey), until the marks cleared.

Phase 6 — Postfactoring (refactor to shorten the propagation)

What this phase is. Postfactoring is refactoring done *after* the change has been actualized — cleaning up the structure now that you have seen how the change rippled. **Why the phase exists:** having lived through that long propagation, you now have concrete evidence of *where* the structure was poor (which methods were duplicated, which were overloaded), so this is the ideal moment to restructure so the *next* owner-related change is cheaper. The deck asks: *could we have made it shorter?* and answers **yes** — **refactoring shortens change propagation** by *moving code affected by the change into fewer classes* and by *splitting the roles of overloaded methods* (Drawlets p.31). Two concrete refactorings — the named "Drawlets change steps" — are studied:

(a) Move the code (Extract + Pull Up Method) [Fowler99]. *What it is:* every class that creates figures has `basicNewFigure(...)`, which does two things — *create a new figure*, and *if it was created at the wrong location, move it* (Drawlets p.33) — so the "move" logic is **duplicated across every creation tool**. *What's done:* the duplicated move logic is **extracted into a new method `moveFigure(...)` and pulled up into the base class `ConstructionTool`** (Drawlets p.33), so it now lives in *one* place that all creation tools inherit. *What it's for / the effect on the change* (the **Refactoring impact** diagram, p.34): the owner-check now needs to touch only `ConstructionTool` (the new home of the logic) plus the four concrete creation tools

`RectangleTool / EllipseTool / RectangularCreationTool / PG_RectImageTool` — a *much smaller* impacted set than the full propagation of p.24, because consolidating the code consolidated the change.

(b) Split the roles. *What it is:* a single method is doing *two different jobs* with the same code, and *propagating the change highlights the difference* because *only one of the roles needs updating* (Drawlets p.35). Concretely, `move(...)` in `AbstractFigure` is used two ways (Drawlets p.36): (i) to move a figure as *requested by the user* — which **must check user identity** (the owner rule applies), and (ii) as *part of new-figure creation* — which **does not need an identity check** (the system, not a user, is positioning the figure). *What's done:* the refactoring **splits** `move(...)` into `move(...)` and `secureMove(...)` so the owner-check lives *only* in the secure variant. *What it's for:* the creation path keeps calling the cheap, unchanged `move`, so it *stays untouched* and drops out of the propagation entirely — separating the two roles means only the role that actually needs the change gets changed, shortening the ripple (Drawlets p.35–36).

Phase 7 — Conclusion (commit / baseline / release)

What this phase is. Conclusion is where the finished owner-change leaves the developer's hands and rejoins the shared world — the same three-step machinery covered in Half 1, now applied to *this* change. **Why the phase exists:** the owner feature is useless until it is integrated with everyone else's work, certified safe, accepted by stakeholders, and delivered; this phase is what does all four.

What was done here. Per the Conclusion deck (Half 1), the finished owner-change is now **committed** to the CM repository (resolving conflicts with anything else committed meanwhile), tested as a **new baseline** (the integrated whole is checked for progress-not-regression, overnight, by the specialised testing team), and eventually **released** to users — small fixes shipping as merged patches, larger ones as installable releases (Conclusion p.2–10). Stakeholders then run **acceptance testing** and **approve the owner feature for release** (Conclusion p.8) — the external sign-off that the feature is what was wanted. The Drawlets deck supplies the concrete *acceptance-test machinery* for this step: acceptance tests were built with a **record/playback tool for Java applications**, and the **functional tests are run with JGiven and Mockito** (Drawlets p.27) — i.e. behaviour-style functional verification feeding the stakeholders' acceptance decision.

Phase 8 — Verification (tests, throughout and after)

What this phase is. Verification is not a single step at the end but the *vertical band* that wraps **every** phase of the change — concretely realised as the **test suite** that is run, extended, and maintained throughout. **Why it exists / what it's for:** it is the continuous guarantee that the change does what the request asked *and* that nothing previously working has broken; it is also where much of a change's real cost lands (the test code changed *more* in percentage terms than the production code). It is listed last only because the deck reports its *numbers* at the end, not because testing happens last. Concretely it comprises:

- **Unit tests: 385 unit tests, 1369 assertions, ~4800 LOC** of unit-test code (Drawlets p.25).
- **Functional tests:** scenario-style cases — *Draw* (figure drawn, for all figures), *Select* (figure selected), *Move* (figure moved), ... — **141 functional test cases** total (Drawlets p.26), executed via **JGiven + Mockito** with a record/playback tool (Drawlets p.27).
- **What actualization did to the tests** (Drawlets p.28): added **new unit tests for the new classes** (`OwnerIdentity`, `SimpleListener`), added **new functional tests for the new functionality** (owner enforcement), and **updated the old tests impacted by the change** during propagation.
- **Test-suite maintenance** (Drawlets p.29): tests from the old version *not affected* by the change are **kept as regression tests**; **obsolete tests are removed**; **tests of new features are added** after the change. (This is the test-side analogue of change propagation.)

- **Results** (Drawlets p.30): production baseline **17 800 LOC**, unit-test baseline **4 800 LOC**; the change **modified 91 production LOC (0.5 %)** and **124 test LOC (2.5 %)** → **≈1.4 test-LOC per modified production-LOC**. The test code changed *more* (in %) than the production code — verification is where much of the change's weight actually lands.

The numerical comparison (refactoring pays off) and final lesson

The closing table compares the three strategies on the *same* owner change (Drawlets p.37):

Metric	No refactoring	Move function	Splitting
Classes added	2	2	2
Interfaces modified	1	1	1
Classes modified	13	8	5
LOC modified	91	95	87

Refactoring barely changes the *lines* edited (91 → 95 → 87) but **roughly halves the number of classes touched** (13 → 8 → 5) — i.e. it shortens the *propagation*, which is the real cost (Drawlets p.37).

Conclusions (Drawlets p.38): *splitting a method into two similar methods may require the least work* — but with two **minuses**: a **new set of unit tests must be created (test code is duplicated)**, and **more effort is needed to create new tests than to adapt existing tests** to a changed implementation. So "fewest classes modified" is not automatically "least total effort" once test cost is counted.

The search path in full — a verified replay of Drawlets p.9–15 (two wrong turns, two backtracks)

The deck actually contains **two** wrong-way/backtrack pairs, not one — the slide titles alternate *Wrong way* (p.10), *Backtrack* (p.11), *Right way* (p.12), *Wrong way* (p.13), *Backtrack* (p.14), *Summary* (p.15). The verified frame-by-frame replay:

1. **p.9 "Locating figure properties"**. Only `SimpleApplet` is green — it is the single comprehended class the search owns at the start (the other three start-set classes are not yet relevant to *this* question). The question on the table: where do *figure properties* live?
2. **p.10 "Wrong way" (first wrong turn — into the palettes)**. `SimpleApplet` turns **orange** (currently examined); `StylePalette`, `ToolBar`, `ToolPalette` turn **green** (pulled into the inspection set); a thick **violet arrow** points *downward* from `SimpleApplet` toward `ToolBar` / `ToolPalette`. Hypothesis under test: figure properties are managed by the palette/UI branch that `SimpleApplet` owns. They are not — palettes hold *styles and tool buttons*, not per-figure state.
3. **p.11 "Backtrack" (first backtrack)**. `ToolPalette` turns **grey** — the palette branch is abandoned as a dead end; `SimpleApplet` stays orange (the search returns to it as the last good position).
4. **p.12 "Right way" (the productive move)**. A thick **blue arrow** runs from `SimpleApplet` *rightward* to the `DrawingCanvas` interface (its circle shown green/marked): the canvas dependency, not the palette dependency, is the road toward figures.
5. **p.13 "Wrong way" (second wrong turn — into the container implementation)**. The `DrawingCanvas` circle turns **orange** (now the examined element), and a thick **violet arrow** drops from it onto `SimpleDrawingCanvas`, which turns **green** while examined. Hypothesis under test: the canvas *implementation* stores figure properties. It does not — the canvas *holds* figures (and a green `SequenceOfFigures` circle above hints at where the holding happens), but a figure's own properties are not the container's business.

6. **p.14 "Backtrack" (second backtrack).** `SimpleDrawingCanvas` turns **grey** — the container branch is abandoned; the search stands at the orange `DrawingCanvas` interface again.
7. **p.15 "Summary" (the located concept).** The full successful path is drawn as a chain of thick blue arrows through **orange waypoints**: `SimpleApplet` (orange) → `DrawingCanvas` (orange circle) → up to `SequenceOfFigures` (orange circle) → up to `Figure` (orange circle) → right to `AbstractFigure`, which alone turns **red**: the class that holds figure properties and must therefore carry the new owner. The two grey boxes (`ToolPalette`, `SimpleDrawingCanvas`) remain as scars of the two failed hypotheses; `StylePalette` and `ToolBar` remain green (inspected en route, harmless).

Three exam-grade observations fall out of the verified replay. First, **both wrong turns go downward** (into the UI branch, into the implementation class) and **the right path goes upward** through the interface chain `DrawingCanvas` → `SequenceOfFigures` → `Figure` to the abstraction `AbstractFigure` — the cleanest possible illustration that per-instance properties live with the *abstraction that defines instances*, not with the *containers or front-ends that use them*. Second, the successful path runs almost entirely **through interfaces** (three of the four waypoint circles), showing that interfaces are the navigational highways of a well-patterned framework. Third, the search needed **seven slides for five hops** — error and recovery are budgeted into the method, not exceptional.

Verified colour tally for every propagation slide (Drawlets p.16–24)

For revision (and for any "describe what slide N shows" question), the exact verified state of each actualization/propagation slide:

- **p.16 "Actualization".** Red: `AbstractFigure`, `SimpleDrawingCanvas` (the two initially edited classes). Blue boxes: `OwnerIdentity`, `SimpleListener` — with a thick **blue arrow** from `OwnerIdentity` up into `AbstractFigure` and a thick **orange arrow** from the `SimpleListener` side into `SimpleDrawingCanvas`, showing where each new class plugs in. Everything else is white: the propagation has not started.
- **p.17 "Change propagation".** Red: `AbstractFigure`, `SimpleDrawingCanvas`. The thick blue arrow now runs `AbstractFigure` → `Figure` and the thick orange arrow `SimpleDrawingCanvas` → `DrawingCanvas`: each changed class throws suspicion onto the interface it implements, and both interface circles turn **green** (marked). This is the canonical picture of "a changed class makes its neighbours inconsistent."
- **p.18 "Propagation – 1".** Red: `AbstractFigure`, `SimpleDrawingCanvas`. Newly green (marked for inspection): `SimpleApplet`, `CanvasTool`, and the `SequenceOfFigures` circle; the `DrawingCanvas` circle is now **orange** (being worked) with thick orange arrows fanning from it to `SimpleApplet`, `CanvasTool` and up to `SequenceOfFigures` — the canvas interface's clients are pulled into the worklist.
- **p.19 "Propagation – 2".** `SimpleApplet` turns **red** (it genuinely changes — it owns the session and the ID button). Thick orange arrows fan from it to `StylePalette`, `ToolBar`, `ToolPalette`, all three now **green** (marked). `CanvasTool` and the `SequenceOfFigures` circle turn **grey** (inspected, no change needed). `Figure` circle still green; `DrawingCanvas` still orange.
- **p.20 "Propagation – 3".** The three palettes settle to **grey** (inspected, unchanged — the style/tool UI does not care about owners). Red set so far: `AbstractFigure`, `SimpleDrawingCanvas`, `SimpleApplet`. The worklist is momentarily almost empty — and then the interface edit lands.
- **p.21 "Propagation – 4".** The **Figure interface circle turns red** — the single "interface modified" of the numerical table (p.37). Thick blue arrows fan from it to `LocatorConnectionHandle` (green), the `Locator` circle (green), `StylePalette` (green again — re-marked!), `SelectionTool`, `ConstructionTool`, `LabelTool` (all green), and the `SequenceOfFigures` circle (green again). `AbstractFigure` renders **pink/violet** on this slide (already-changed, settled). Grey: `ToolBar`, `ToolPalette`, `CanvasTool`. This is the largest one-step fan-out of the whole change, and it includes the instructive detail that a class already

dismissed (`StylePalette`, grey on p.20) is *re-marked* when a different neighbour (the `Figure` interface) changes — being cleared once is not immunity.

- **p.22 "Propagation – 5"**. The marks resolve into changes: red now also covers `LocatorConnectionHandle`, `StylePalette`, `SelectionTool`, `ConstructionTool`, `PrototypeConstructionTool` (and `AbstractFigure` is red again). Green (just marked): `ShapeTool`, `LabelTool` — with a thick blue arrow `ConstructionTool` → `LabelTool` and a double-headed thick blue arrow between `PrototypeConstructionTool` and `ShapeTool` marking `ConstructionTool`'s subclasses for inspection. Grey: `Locator` circle, `SequenceOfFigures` circle, `ToolBar`, `ToolPalette`, `CanvasTool`.
- **p.23 "Propagation – 6"**. `LabelTool` turns red; `ShapeTool` turns orange (the current frontier); thick blue arrows run along the bottom row marking all four leaf creation tools — `RectangleTool`, `EllipseTool`, `RectangularCreationTool`, `PG_RectImageTool` — green for inspection.
- **p.24 "Propagation – done"**. The final state (see the reconciliation below): all four leaf tools red, `ShapeTool` left orange, no green marks remain anywhere — the worklist is empty and the propagation is complete.

Reconciling the final diagram (p.24) with the numbers table (p.37)

The "Propagation – done" slide and the "Numerical data" table corroborate each other exactly, and being able to show that is a high-value exam move:

- **Red boxes on p.24 — the 13 modified classes:** `LocatorConnectionHandle`, `AbstractFigure`, `StylePalette`, `SimpleApplet`, `SimpleDrawingCanvas`, `SelectionTool`, `ConstructionTool`, `PrototypeConstructionTool`, `LabelTool`, `RectangleTool`, `EllipseTool`, `RectangularCreationTool`, `PG_RectImageTool`. Count: **13** — precisely the "Classes modified: 13" of the no-refactoring column (Drawlets p.24, p.37).
- **Red circle on p.24 — the 1 modified interface:** `Figure` — precisely "Interfaces modified: 1" (Drawlets p.24, p.37).
- **Blue boxes on p.24 — the 2 added classes:** `OwnerIdentity`, `SimpleListener` — precisely "Classes added: 2" (Drawlets p.16, p.24, p.37).
- **Grey on p.24 — visited but unchanged:** `ToolBar`, `ToolPalette`, `CanvasTool` (boxes) and `Locator`, `SequenceOfFigures` (interface circles). These five are the *measure of inspection overhead*: real propagation work was spent on them even though they contribute zero to "classes modified" — more evidence that propagation cost ≠ edit cost.
- **Orange on p.24 — visited with attention, not modified:** `ShapeTool` (box) and the `DrawingCanvas` circle. `ShapeTool` is the deck's subtlest detail: *all four of its subclasses* changed, and `ShapeTool` itself sits orange between red `ConstructionTool` above and four red leaves below — the abstract middle layer was traversed and scrutinised but did not itself need the owner check (its `basicNewFigure` duplication lives in the leaves, which is exactly what the moving-the-code refactoring later fixes, p.33–34).
- **p.32 "Propagation"** repeats p.24 with all transient marker colours cleared (only red/blue remain, everything else white) — it is the clean "no refactoring" reference picture that the refactoring discussion of p.33–36 then improves upon.

The refactoring-impact diagram decoded (Drawlets p.34)

The refactoring-impact slide drops the red/green state machine entirely and uses a fresh two-colour scheme on an otherwise all-white graph: `ConstructionTool` in yellow and the four leaf creation tools — `RectangleTool`, `EllipseTool`, `RectangularCreationTool`, `PG_RectImageTool` — in cyan (Drawlets p.34). Reading: after extracting the duplicated wrong-location move logic out of each tool's `basicNewFigure(...)` into the new method `moveFigure(...)` and making it a member of the base class `ConstructionTool`

(Drawlets p.33), the code affected by the owner concern now lives in the one yellow class, and the four cyan classes are reduced to mere call sites of the inherited method. The contrast with p.24/p.32 is the slide's whole argument: the same concern that previously spattered red across the entire upper graph now colours a five-class pocket at the bottom of the tool tree — and the numerical column "Move function: classes modified 8" (down from 13) is that picture in numbers (Drawlets p.34, p.37).

Definitions & Terminology

Term	Definition	Source
Conclusion (phase)	The last phase of a software change — the hand-off machinery (commit → baseline → release) that folds a finished change back into the shared codebase and out to users; concrete activities depend on the specific software process. Used to turn a private edit into shared, certified, delivered software.	Conclusion p.1
Commit	The step where programmers return updated code to the configuration-management repository, resolving conflicts with others' concurrent commits. Used to make a change visible to the whole team and force merge conflicts to be resolved now.	Conclusion p.2
Baseline	A thoroughly-tested, agreed-upon snapshot of the <i>whole</i> system that everyone builds on next; must represent progress, never a regression on already-working functionality. Used as the team's trustworthy "known-good" reference and synchronisation point.	Conclusion p.2–3
Baseline testing	The thorough testing that certifies a candidate snapshot into baseline status; expensive (overnight/weekend, run by a specialised team) and scheduled, not per-commit. Used to prove the <i>integrated whole</i> works, not just one file in isolation.	Conclusion p.3
Deadline to commit	The moment baseline testing starts; any change not committed by then misses this baseline and must re-integrate against the next (moved) one, with management tracking misses. Used as both a synchronisation point and an accountability mechanism.	Conclusion p.5
Bugs in baseline	The <i>minor</i> -severity verdict: faults found in testing are small enough that the testing team still certifies the baseline and pushes the fixes onto the bug-report stack for future changes. Used to keep progress moving rather than stalling over low-impact defects.	Conclusion p.6
Broken baseline	The <i>serious</i> -severity verdict: faults are bad enough that the testing team rejects the buggy commits and possibly the whole increment, so no new baseline is created and the work is invalidated/postponed; offending programmers are named. Used to protect the team from building on a rotten foundation.	Conclusion p.7
Acceptance testing	Functional testing performed by the <i>stakeholders</i> (not the dev/testing team) that thoroughly exercises features in user terms. Used both to inform stakeholders of progress and as the external sign-off gate that approves software for release — the layer after internal baseline certification.	Conclusion p.8
Release	The delivery of baseline code to <i>users</i> — the only step that crosses out of the organisation; requires substantial extra work and its frequency is a technical <i>and</i> business decision. Used to actually put the change in users' hands.	Conclusion p.9
Patch / merge	A <i>small</i> release delivered as a patch and incorporated into the already-installed program by the tool "merge," so only changed pieces are applied. Used to ship fixes/minor features without a full reinstall (contrast: large releases are downloaded and installed whole).	Conclusion p.10
Versioned model of software lifespan	A release scheme mixing infrequent large releases (major version bump) with frequent small ones (minor bump), e.g. "AwesomeApp 4.2." Used to balance delivering value often against the cost/risk of each release.	Conclusion p.9

Term	Definition	Source
Drawlets	Application framework adding a drawing canvas to a host app; > 100 classes, 35 interfaces, ~40 000 LOC; by Beck & Cunningham, ported to Java, a self-described "perfect API." Used in this lecture as the L10 case study for one end-to-end change.	Drawlets p.1–2
SimpleApplet	The bundled host application for Drawlets; provides the canvas/toolbars/buttons and runs in any browser. Used here both as the comprehension starting point for concept location and as the session/ID manager the owner change extends.	Drawlets p.3
AbstractFigure	The class holding per-figure properties (where each figure's own state lives); the concept-location result for "figure" and the origin where the owner property attaches and the change begins.	Drawlets p.15–16
Concept classification	Sorting change-request concepts into irrelevant / external / significant. Used to decide <i>which</i> concept to anchor the code search on — here figure (significant), not owner (external).	Drawlets p.8
Backtracking (concept location)	Abandoning a wrong candidate class and returning to the last good one to try another path. Used because concept location is a <i>search</i> (you went to the container SimpleDrawingCanvas , then backtracked to the abstraction AbstractFigure), not a lookup.	Drawlets p.10–14
Change propagation	Visiting each class made inconsistent by a change and changing-or-confirming it (which can make <i>its</i> neighbours inconsistent) until all marks clear. Used to keep the system consistent and to measure a change's true cost (classes touched).	Drawlets p.16–24
OwnerIdentity / SimpleListener	The two new classes added by the change: OwnerIdentity represents the user identity an owner carries; SimpleListener is an Observer that wires that identity into the figure/canvas machinery. Used to introduce and connect the external "owner" concept with minimal new structure.	Drawlets p.16
moveFigure(...)	A method extracted from the duplicated move-logic in basicNewFigure(...) and pulled up into base class ConstructionTool . Used (Move-the-code refactoring) to consolidate the affected code into one place so the owner change touches far fewer classes.	Drawlets p.33
move / secureMove	The result of role-splitting AbstractFigure.move(...) : secureMove (user-requested move) checks owner identity, plain move (creation-time move) does not. Used (Split-the-roles refactoring) so only the role that needs the owner-check is changed and the creation path stays untouched.	Drawlets p.35–36
Regression tests	Old tests <i>unaffected</i> by the change, kept to guard future changes (obsolete tests removed, new-feature tests added). Used to ensure the change is "progress, not regression" — that previously-working behaviour still works.	Drawlets p.29
JGiven / Mockito	Tools used to run Drawlets' functional/acceptance tests (alongside a record/playback tool for Java apps). Used to drive the behaviour-style functional verification that feeds the stakeholders' acceptance decision.	Drawlets p.27

Additional terminology from the verified figures and slide text

Term	Definition	Source
Change mini-process (phase diagram)	The sequence Initiation → Concept Location → Impact Analysis → Prefactoring → Actualization → Postfactoring → Conclusion, with Verification drawn as a vertical band spanning all phases; Conclusion is highlighted as the last phase.	Conclusion p.1
Configuration management repository	The shared store programmers return ("commit") their updated code to at the start of the conclusion phase, resolving conflicts as they do.	Conclusion p.2

Term	Definition	Source
Specialized testing team	The team (distinct from the developers) that conducts baseline testing, decides whether bugs are minor (certify anyway) or serious (reject commits), and can identify which programmers committed buggy files.	Conclusion p.3, p.6–7
Stack of bug reports	Where corrections for <i>minor</i> baseline bugs are recorded; they are fixed as part of future changes rather than blocking certification.	Conclusion p.6
"AwesomeApp 4.2"	The deck's illustrative product version under the versioned model of software lifespan: infrequent large releases move the major number, frequent small releases the minor.	Conclusion p.9
Tool "merge"	The named mechanism by which small releases (patches) are incorporated into the user's installed program, as opposed to downloading and installing a large release whole.	Conclusion p.10
Figure (interface)	The central interface of the Drawlets figure hierarchy (drawn as a lollipop circle); implemented by AbstractFigure ; the single interface modified by the owner change — its modification triggers the largest fan-out of the propagation.	Drawlets p.5, p.21, p.37
DrawingCanvas (interface)	The canvas abstraction (lollipop circle) connecting SimpleApplet , CanvasTool , SimpleDrawingCanvas and SequenceOfFigures ; repeatedly the orange "being worked" hub during propagation but never counted as modified.	Drawlets p.5, p.17–24
SequenceOfFigures (interface)	The interface (lollipop circle) through which the canvas holds its figures, sitting between Figure and DrawingCanvas on the successful search path; inspected during propagation, unchanged (grey at p.24).	Drawlets p.5, p.15, p.24
Locator / LocatorConnectionHandle	Locator is an interface (circle) used by LocatorConnectionHandle and related to Figure ; LocatorConnectionHandle is a client class of Figure that <i>did</i> require modification (red at p.24) once the Figure interface changed.	Drawlets p.5, p.21–24
SimpleDrawingCanvas	The concrete canvas implementation. Plays two roles in the lecture: the <i>second wrong-way candidate</i> of concept location (examined and backtracked, grey at p.14–15) and one of the <i>two initially edited classes</i> of actualization (red from p.16, wired to the new listener).	Drawlets p.13–16
CanvasTool / SelectionTool / ConstructionTool	The tool hierarchy: CanvasTool (base, under DrawingCanvas) with subclasses SelectionTool and ConstructionTool ; ConstructionTool 's subclasses are PrototypeConstructionTool , ShapeTool , LabelTool . CanvasTool ends the change grey (unchanged); SelectionTool , ConstructionTool , PrototypeConstructionTool , LabelTool end red (modified).	Drawlets p.5, p.24
ShapeTool and its four subclasses	ShapeTool (abstract middle layer, orange/unmodified at p.24) parents the four concrete creation tools RectangleTool , EllipseTool , RectangularCreationTool , PG_RectImageTool — all four modified by the owner change and all four cyan beneficiaries of the moving-the-code refactoring.	Drawlets p.5, p.23–24, p.34
basicNewFigure(...)	The method present in <i>each</i> class that creates new figures, with two main parts: create a new figure, and if the figure was created at the wrong location, move it. Its duplicated second part across every creation tool is the raw material for the moveFigure(...) extraction.	Drawlets p.33

Term	Definition	Source
Lollipop notation	The UML convention used throughout the deck's class diagrams: interfaces drawn as small named circles, classes as rectangles, inheritance as hollow-triangle arrows. Knowing it is what lets you count "1 interface modified" off the final diagram.	Drawlets p.5, p.24
Record/playback tool	The kind of tool (for Java applications) used to <i>create</i> the acceptance tests, capturing interactive sessions for replay; distinct from JGiven/Mockito, which <i>run</i> the functional tests.	Drawlets p.27
"Perfect API"	RoleModel Software's marketing description of Drawlets (the deck cites rolemodelsoft.com/aboutUs/drawlets.htm) — quoted by the lecture with mild irony, since even a "perfect" API exhibits a 13-class propagation for a small feature.	Drawlets p.2

Common Pitfalls / Gotchas

- **"Commit = done."** No — commit only hands code to the *team*. It must still pass **baseline testing** and (for users) be **released**; each is a wider gate (Conclusion p.2, p.8–9).
- **Confusing "bugs in baseline" with "broken baseline."** Minor bugs → baseline *still certified*, fixes deferred to the stack; serious bugs → commits *rejected*, increment possibly *invalidated*. The difference is severity and has schedule/reputation consequences (Conclusion p.6–7).
- **Thinking baseline frequency is free either way.** Too rare → bug accumulation; too frequent → overhead. It is a deliberate trade-off, not "more testing is always better" (Conclusion p.4).
- **Treating the deadline as soft.** Missing it is not neutral: it costs *re-integration work against a moved baseline* and is *visible to management* (Conclusion p.5).
- **Concept location = look it up.** It is a **graph search with backtracking**; expect a wrong turn. On Drawlets the wrong move was going to the *container* (`SimpleDrawingCanvas`) instead of the *abstraction* (`AbstractFigure`) (Drawlets p.10–15). Dependency *direction* (up to the abstraction vs. down to the container) decides correctness.
- **Skipping concept classification.** Without sorting concepts into irrelevant/external/significant you don't know *which* concept to anchor the search on — here, `figure` (significant), not `owner` (external) (Drawlets p.8).
- **Equating "lines changed" with "cost."** Drawlets modified only **0.5 % of production LOC** yet propagated across **~13 classes**. Cost = propagation size, not edit size (Drawlets p.30, p.37).
- **Assuming "fewest classes modified" = "least effort."** Splitting touches the fewest classes (5) but **duplicates test code** and **costs more to write new tests** than to adapt old ones — total effort can be higher (Drawlets p.37–38).
- **Forgetting the test suite during propagation.** Tests propagate too: update impacted tests, add tests for new classes/features, keep unaffected tests as regression, delete obsolete ones (Drawlets p.28–29).
- **Ignoring refactoring because "the change already works."** A long propagation is a *signal* to refactor (move code into fewer classes, split roles) so the *next* change is cheaper (Drawlets p.31).

Additional traps verified against the figures

- **"There was one wrong turn."** There were **two**: first into the palette branch (`ToolPalette` greyed at the first Backtrack, p.10–11), then into the container implementation (`SimpleDrawingCanvas` greyed at the second Backtrack, p.13–14). The slide titles literally alternate Wrong way / Backtrack / Right way / Wrong way / Backtrack / Summary (Drawlets p.10–15).
- **"The located class was found directly from `SimpleApplet`."** The successful path has *four hops* through *three interfaces*: `SimpleApplet` → `DrawingCanvas` → `SequenceOfFigures` → `Figure` →

`AbstractFigure` (Summary slide, Drawlets p.15). Quoting the full chain is what distinguishes a top answer.

- **"ShapeTool was modified."** It was not — `ShapeTool` ends the change **orange** (traversed, scrutinised, unchanged) even though its parent `ConstructionTool` and *all four* of its subclasses are red. Including it would make the class count 14 and contradict the table's 13 (Drawlets p.24, p.37).
- **"The modified interface is `DrawingCanvas`."** The one modified interface is `Figure` (the only circle that turns red, p.21–24); `DrawingCanvas` stays orange and `Locator / SequenceOfFigures` end grey (Drawlets p.24, p.37).
- **"Once a class is checked it stays checked."** `StylePalette` was dismissed grey on p.20, then **re-marked green on p.21** when the `Figure` interface changed, and ended **red** (modified) — a class must be re-examined every time *another* of its neighbours changes (Drawlets p.20–24).
- **"The change started in one class."** Actualization begins with **two** edited classes — `AbstractFigure` and `SimpleDrawingCanvas` — plus the two new ones; both red classes independently throw inconsistency onto their respective interfaces (`Figure` , `DrawingCanvas`) on p.17 (Drawlets p.16–17).
- **"session is classified irrelevant in the table."** On the slide as printed, the `session` row has **no mark in any column** — every other concept has exactly one 'x'. If asked to reproduce the table, reproduce the omission or flag it (Drawlets p.8).
- **"Acceptance tests and functional tests are the same artefact in the Drawlets deck."** The acceptance tests were *created* with a record/playback tool for Java applications; the functional tests are *run* with JGiven and Mockito — two different tool sentences on the same slide (Drawlets p.27).
- **"Baseline testing is done by the developers."** The Conclusion deck assigns it to a *specialized testing team*, which is also the actor that certifies, rejects, and names offending programmers (Conclusion p.3, p.6–7).
- **"A patch is just a small download-and-install."** Small releases are delivered as patches *incorporated into the user's program by the tool "merge"* — the user's installed program is transformed in place, not replaced; download-and-install is the *large-release* mechanism (Conclusion p.10).

Exam Focus

Most-likely questions

1. **List and explain the three steps of the conclusion phase** (commit → new baseline → new release), noting that activities depend on the software process (Conclusion p.1–2). *Have a one-line definition of each ready.*
2. **Contrast "bugs in baseline" vs. "broken baseline"** — what the testing team does in each and the consequences (defer-to-stack vs. reject-and-invalidate; reputation) (Conclusion p.6–7).
3. **Explain baseline testing and the deadline** — why thorough, why expensive, the frequency trade-off, what missing the deadline costs (Conclusion p.3–5).
4. **Walk the Drawlets owner change through all eight phases** — this is the signature L10 essay. Hit: request (owner per figure) → classify concepts (figure/canvas significant; owner/ID/password external) → locate `AbstractFigure` (with the wrong-way/backtrack/right-way story) → impact (whole upper graph) → actualize (`OwnerIdentity` , `SimpleListener` ; propagate class-by-class) → postfactor (move `moveFigure` ; split `move` / `secureMove`) → conclude (commit/baseline/release) → verify (385 unit / 141 functional tests) (Drawlets, whole deck).
5. **Describe concept location as search with backtracking**, using the container-vs-abstraction wrong turn as the example (Drawlets p.9–15).

6. **Describe change propagation as a marking/visiting process** and why one small edit ripples widely (Drawlets p.16–24, p.30).
7. **Explain how refactoring shortens propagation** and read the comparison table: classes modified 13 → 8 → 5 while LOC stays ~91; plus the splitting caveat (test duplication) (Drawlets p.31–38).

Numbers worth memorising (Drawlets): 385 unit tests / 1369 assertions / 4800 test LOC; 141 functional tests; production baseline 17 800 LOC; change = 91 production LOC (0.5 %) + 124 test LOC (2.5 %) ≈ 1.4 test-LOC per production-LOC; classes modified 13 / 8 / 5 for no-refactor / move / split (Drawlets p.25–37).

Key names to drop: `AbstractFigure` (located concept), `OwnerIdentity` + `SimpleListener` (new classes), `moveFigure` (pulled up to `ConstructionTool`), `move` / `secureMove` (role split), JGiven + Mockito (functional tests), Beck & Cunningham (authors), Rajlich Ch. 17 [Raj13] .

Trap to avoid: do *not* say the conclusion is "just committing." Commit is one of three steps, and certification (baseline) + stakeholder acceptance + release are the rest.

Rapid-fire fact sheet (one line per fact, all verified)

- Conclusion = **last phase** of software change; activities **depend on the specific software process** (Conclusion p.1).
- Three steps: **Commit** → **New baseline** → **New release** (Conclusion p.2).
- Commit = return updated code to the **configuration management repository, resolving conflicts** (Conclusion p.2).
- Baseline testing guarantees: **as bug-free as possible + progress, not a regression** (Conclusion p.3).
- Baseline testing is run **overnight or over the weekend** by a **specialized testing team** (Conclusion p.3).
- Baseline frequency depends on **size of the program and required quality**; too rare → **bug accumulation**, too frequent → **unnecessary overhead** (Conclusion p.4).
- Deadline to commit = **the time when testing of the baseline starts**; miss it → submit by the **next** deadline, with **additional (sometimes significant) work**; management **knows how often** a programmer missed and **may require an explanation** (Conclusion p.5).
- Minor bugs → **still certify**; corrections go onto the **stack of bug reports**, fixed in **future changes** (Conclusion p.6).
- Serious bugs → **reject the buggy commits**; possibly **no new baseline**, all work **invalidated or postponed** (a significant cost on large projects); offending programmers identified, **reputation suffers** (Conclusion p.7).
- Stakeholders: acceptance testing = **functional testing done by the stakeholders**; informs them of **progress**; they **approve software for the release** (Conclusion p.8).
- Releases: **substantial extra work**; frequency is a **technical and business decision**; **less frequent large + more frequent small** releases; "**versioned model of software lifespan**"; e.g. **AwesomeApp 4.2** (Conclusion p.9).
- Large releases: **download and install**; small releases: **patches incorporated by the tool "merge"** (Conclusion p.10).
- Drawlets: framework that **adds graphical display to a host application**; canvas figures = lines, free-hand lines, rectangles, rounded rectangles, triangles, pentagons, polygons, ellipses, text boxes, images (Drawlets p.1).
- Drawlets size: **>100 classes, 35 interfaces, 40 000 LOC**; by **Kent Beck and Ward Cunningham**, later **ported into Java**; "**perfect API**" (Drawlets p.2).

- Host app: provides **canvas instance, toolbars, tool buttons**; **SimpleApplet** is part of the Drawlets library and **runs in any browser** (Drawlets p.3).
- Change request: **owner per figure**; owner = user who put the figure onto the canvas; **only the owner may modify**; **ID + password at session start**; owners own all figures created during the session; rationale: more versatile/useful + **cooperative work** (Drawlets p.6).
- Classification: **significant = figure, canvas**; **external = owner, ID, password**; the rest irrelevant (Drawlets p.8).
- Located concept: **AbstractFigure** (red on the Summary slide) via **SimpleApplet** → **DrawingCanvas** → **SequenceOfFigures** → **Figure** (Drawlets p.15).
- New classes: **OwnerIdentity**, **SimpleListener** (Drawlets p.16).
- Final modified set: **13 classes + the Figure interface**; unchanged-but-inspected: **ToolBar**, **ToolPalette**, **CanvasTool**, **Locator**, **SequenceOfFigures**, **ShapeTool** (Drawlets p.24, p.37).
- Tests: **385 unit tests / 1369 assertions / 4800 lines**; **141 functional test cases** (Draw, Select, Move, ...); record/playback acceptance tests; **JGiven + Mockito** (Drawlets p.25–27).
- Test work during actualization: **new unit tests for new classes, new functional tests for new functionality, old impacted tests updated** (Drawlets p.28).
- Test-suite maintenance: keep unaffected as **regression tests**, **remove obsolete**, **add new-feature tests after the change** (Drawlets p.29).
- Results: baseline **17 800 production LOC + 4800 unit-test LOC**; change = **91 production LOC (0.5 %) + 124 test LOC (2.5 %) ≈ 1.4 test lines per production line** (Drawlets p.30).
- Refactoring: **long propagation is a problem** → **shorten it by moving code into fewer classes and splitting the roles** (Drawlets p.31).
- Moving the code: extract **moveFigure(...)** from **basicNewFigure(...)** 's "wrong location → move it" part; make it a member of **ConstructionTool** (Drawlets p.33).
- Splitting the roles: **move(...)** in **AbstractFigure** used (i) for user-requested moves — **must check user identity** — and (ii) during new-figure creation — **no identity check needed**; split into **move(...)** and **secureMove(...)** (Drawlets p.35–36).
- Table: classes added **2/2/2**; interfaces modified **1/1/1**; classes modified **13/8/5**; LOC modified **91/95/87** for no-refactoring / move function / splitting (Drawlets p.37).
- Conclusions: splitting **may require least work**; minus 1 — **new set of unit tests must be created (test code duplicated)**; minus 2 — **creating new tests costs more effort than adapting existing ones** (Drawlets p.38).

Practice questions (with answers)

1. **Q:** Name the seven phases on the Conclusion deck's opening diagram and the element that spans them all.
A: Initiation, Concept Location, Impact Analysis, Prefactoring, Actualization, Postfactoring, Conclusion — spanned vertically by Verification (Conclusion p.1).
2. **Q:** What two guarantees must baseline testing deliver? **A:** That the new baseline is as bug-free as possible, and that it represents progress of the project, not a regression (Conclusion p.3).
3. **Q:** Who conducts baseline testing, and when does it typically run? **A:** A specialized testing team; often overnight or over the weekend because it can take significant time (Conclusion p.3).
4. **Q:** State both halves of the baseline-frequency trade-off. **A:** Postponing too long allows a large accumulation of bugs that makes the testing task difficult; testing too frequently is unnecessary overhead. The chosen frequency depends on program size and required quality (Conclusion p.4).
5. **Q:** Exactly when does the "deadline to commit" fall, and what are all three consequences of missing it? **A:** At the time the testing of the baseline starts. Consequences: submit by the next deadline; additional

(possibly significant) work to fit the new baseline; management knows how often the programmer missed and may require an explanation (Conclusion p.5).

6. **Q:** A committed file has a minor fault discovered in baseline testing. What happens? **A:** The testing team can still certify the updated code as the new baseline; the correction is added to the stack of bug reports and fixed as part of future changes (Conclusion p.6).
7. **Q:** And if the fault is serious? **A:** The testing team can reject the buggy commits; sometimes the whole work done on the new baseline is rejected — no new baseline is created, all work is invalidated or postponed (a significant cost on large projects); the team can identify who committed the buggy files and those programmers' reputation suffers (Conclusion p.7).
8. **Q:** Define acceptance testing per the Conclusion deck and give both of its purposes. **A:** Functional testing done by the stakeholders that thoroughly tests software functionalities; it gives stakeholders information about project progress and is how they approve software for the release (Conclusion p.8).
9. **Q:** Contrast the delivery mechanism of large and small releases. **A:** Large releases are downloaded and installed; small releases are delivered as patches incorporated into the user's program by the tool "merge" (Conclusion p.10).
10. **Q:** Reproduce the change request for the Drawlets example. **A:** Implement an owner for each figure; the owner is the user who put the figure onto the canvas and only the owner should be allowed to modify it; at the beginning of a session users input their ID and password and become owners of all figures created during the session (Drawlets p.6).
11. **Q:** Why was this change worth initiating? **A:** It makes `SimpleApplet` more versatile and useful — support for cooperative work (Drawlets p.6).
12. **Q:** Classify: owner, figure, user, canvas, ID. **A:** owner — external; figure — significant; user — irrelevant; canvas — significant; ID — external (Drawlets p.8).
13. **Q:** Why must the code search anchor on a significant concept rather than an external one? **A:** External concepts (owner, ID, password) do not yet exist in the code — there is nothing to locate; significant concepts (figure, canvas) are already in the design, so the search starts there and the external concepts are attached at the found location (Drawlets p.8, p.16).
14. **Q:** Recount the two wrong turns of concept location and what each teaches. **A:** First into the palette/UI branch from `SimpleApplet` (palettes manage styles and tools, not figure state); second from `DrawingCanvas` down into `SimpleDrawingCanvas` (the container implementation holds figures but not a figure's own properties). Both teach that per-figure properties belong to the figure abstraction, reached by going up the interface chain, not down into containers or UI (Drawlets p.10–15).
15. **Q:** Quote the successful search path in full. **A:** `SimpleApplet` → `DrawingCanvas` → `SequenceOfFigures` → `Figure` → `AbstractFigure` (located, red) (Drawlets p.15).
16. **Q:** Which two classes does actualization edit first, and which two does it add? **A:** Edits `AbstractFigure` and `SimpleDrawingCanvas`; adds `OwnerIdentity` and `SimpleListener` (Drawlets p.16).
17. **Q:** What single event causes the largest one-step fan-out during propagation, and which classes does it mark? **A:** The modification of the `Figure` interface (Propagation – 4): it marks `LocatorConnectionHandle`, `Locator`, `StylePalette` (re-marked after being dismissed), `SelectionTool`, `ConstructionTool`, `LabelTool`, and `SequenceOfFigures` (Drawlets p.21).
18. **Q:** List the 13 modified classes. **A:** `LocatorConnectionHandle`, `AbstractFigure`, `StylePalette`, `SimpleApplet`, `SimpleDrawingCanvas`, `SelectionTool`, `ConstructionTool`, `PrototypeConstructionTool`, `LabelTool`, `RectangleTool`, `EllipseTool`, `RectangularCreationTool`, `PG_RectImageTool` (Drawlets p.24, p.37).
19. **Q:** Which inspected elements ended the change unmodified? **A:** Classes `ToolBar`, `ToolPalette`, `CanvasTool`, `ShapeTool`; interfaces `Locator`, `SequenceOfFigures`, `DrawingCanvas` (Drawlets p.24).

20. **Q:** Give the unit-test and functional-test inventory of Drawlets. **A:** 385 unit tests, 1369 assertions, 4800 lines of unit-test code; 141 functional test cases of the form Draw (figure drawn on the canvas, for all figures), Select (figure is selected), Move (figure is moved to a different location), etc. (Drawlets p.25–26).
21. **Q:** What three kinds of test work did actualization entail? **A:** New unit tests for the new classes; new functional tests for the new functionality; updating the old tests impacted during change propagation (Drawlets p.28).
22. **Q:** State the three rules of test-suite maintenance. **A:** Keep old-version tests unaffected by the change as regression tests for the future; remove obsolete tests; add tests of the new features after the change (Drawlets p.29).
23. **Q:** Give all four "Results" numbers and the derived ratio. **A:** Production baseline 17 800 lines; unit-test baseline 4800 lines; 91 production lines modified (0.5 %); 124 test lines modified (2.5 %); ≈ 1.4 test lines per modified production line (Drawlets p.30).
24. **Q:** Describe both propagation-shortening refactorings with their Drawlets instantiation. **A:** (1) Move code affected by change into fewer classes: each figure-creating class's `basicNewFigure(...)` both creates a figure and moves it if created at the wrong location; the move part was extracted as `moveFigure(...)` and made a member of base class `ConstructionTool`. (2) Split the roles: `move(...)` in `AbstractFigure` serves user-requested moves (must check identity) and creation-time moves (needn't); it was split into `move(...)` and `secureMove(...)` (Drawlets p.31, p.33, p.35–36).
25. **Q:** Reproduce the numerical-data table. **A:** Classes added 2 / 2 / 2; interfaces modified 1 / 1 / 1; classes modified 13 / 8 / 5; LOC modified 91 / 95 / 87 — columns: no refactoring, move function, splitting (Drawlets p.37).
26. **Q:** Why might splitting *not* be the cheapest option despite touching only 5 classes? **A:** A new set of unit tests must be created, duplicating test code, and creating new tests requires more effort than adapting existing tests to a changed implementation (Drawlets p.38).
27. **Q:** Which lecture-wide claim do the 91-LOC / 13-class numbers prove? **A:** That the cost of a change is driven by the size of its propagation (classes visited and reconciled), not by the number of lines edited — 0.5 % of the code rippled through roughly the whole upper class graph (Drawlets p.24, p.30, p.37).

The Drawlets Class Graph — Complete Structural Reference

This section reconstructs the *entire* top-classes diagram (Drawlets p.5) as text, so it can be searched and reproduced without the figure. The diagram recurs — with changing colour states — on p.9–24 and p.32; p.34 uses the same graph for the refactoring impact.

Inventory

Interfaces (lollipop circles), 4 shown of 35 total (Drawlets p.2, p.5): `Locator`, `Figure`, `SequenceOfFigures`, `DrawingCanvas`.

Classes (boxes), 18 shown of >100 total (Drawlets p.2, p.5): `LocatorConnectionHandle`, `AbstractFigure`, `StylePalette`, `SimpleApplet`, `ToolBar`, `ToolPalette`, `CanvasTool`, `SimpleDrawingCanvas`, `SelectionTool`, `ConstructionTool`, `PrototypeConstructionTool`, `ShapeTool`, `LabelTool`, `RectangleTool`, `EllipseTool`, `RectangularCreationTool`, `PG_RectImageTool` — plus, from actualization onward, the two added classes `OwnerIdentity` and `SimpleListener` (Drawlets p.16).

Edges and hierarchies (verified from the figure)

- **Host cluster (left column):** `SimpleApplet` connects upward to `StylePalette` and downward to `ToolBar` and `ToolPalette` — the four classes a user sees on screen, and the green comprehension starting set (Drawlets p.4–5).
- **Figure cluster (top):** `LocatorConnectionHandle` uses the `Locator` interface; both `Locator` and `AbstractFigure` connect to the central **Figure** interface, into which arrows converge from all over the graph (it is visibly the most-depended-upon node). `AbstractFigure` implements `Figure` (Drawlets p.5).
- **Canvas spine (centre):** beneath `Figure` sits the `SequenceOfFigures` interface, and beneath it the `DrawingCanvas` interface; `SimpleApplet` points into `DrawingCanvas` from the left; `CanvasTool` and `SimpleDrawingCanvas` hang beneath `DrawingCanvas` (Drawlets p.5).
- **Tool inheritance tree (bottom):** `CanvasTool` is specialised (hollow-triangle inheritance) by `SelectionTool` and `ConstructionTool`; `ConstructionTool` is specialised by `PrototypeConstructionTool`, `ShapeTool`, and `LabelTool`; `ShapeTool` is specialised by the four leaf creation tools `RectangleTool`, `EllipseTool`, `RectangularCreationTool`, `PG_RectImageTool` (Drawlets p.5).
- **Added wiring (from p.16):** `OwnerIdentity` connects up into `AbstractFigure` (the figure carries its owner's identity) and is fed by `SimpleListener`, which also wires into `SimpleDrawingCanvas` — the listener channels session identity from the canvas side into the figures (Drawlets p.16).

Fate of every node under the owner change (the master table)

Node	Kind	Concept-location role	Propagation fate (p.24)	In "13 modified"?
<code>SimpleApplet</code>	class	search start (green p.9, orange p.10–15)	red — manages session/ID	yes
<code>StylePalette</code>	class	inspected en route (green)	red — dismissed grey p.20, re-marked p.21, modified	yes
<code>ToolBar</code>	class	inspected en route (green)	grey — unchanged	no
<code>ToolPalette</code>	class	first wrong way; greyed at first backtrack	grey — unchanged	no
<code>Locator</code>	interface	—	grey — inspected, unchanged	no (interface)
<code>LocatorConnectionHandle</code>	class	—	red — client of changed <code>Figure</code>	yes
<code>Figure</code>	interface	waypoint on the right way (orange p.15)	red — the 1 modified interface	no (counted as interface)
<code>AbstractFigure</code>	class	located concept (red, p.15)	red — origin of the change	yes
<code>SequenceOfFigures</code>	interface	waypoint on the right way (orange p.15)	grey — inspected, unchanged	no (interface)
<code>DrawingCanvas</code>	interface	waypoint on the right way (orange p.15)	orange — repeatedly worked hub, unmodified	no (interface)
<code>SimpleDrawingCanvas</code>	class	second wrong way; greyed at second backtrack	red — second initially-edited class	yes
<code>CanvasTool</code>	class	—	grey — unchanged	no
<code>SelectionTool</code>	class	—	red	yes

Node	Kind	Concept-location role	Propagation fate (p.24)	In "13 modified"?
ConstructionTool	class	—	red (and yellow target of <code>moveFigure</code> pull-up, p.34)	yes
PrototypeConstructionTool	class	—	red	yes
ShapeTool	class	—	orange — traversed, unmodified	no
LabelTool	class	—	red	yes
RectangleTool	class	—	red (cyan on p.34)	yes
EllipseTool	class	—	red (cyan on p.34)	yes
RectangularCreationTool	class	—	red (cyan on p.34)	yes
PG_RectImageTool	class	—	red (cyan on p.34)	yes
OwnerIdentity	class (new)	— (external concept made code)	blue — added	counted in "2 added"
SimpleListener	class (new)	—	blue — added	counted in "2 added"

Row count check: thirteen "yes" rows = "Classes modified: 13"; one red interface = "Interfaces modified: 1"; two blue rows = "Classes added: 2" (Drawlets p.24, p.37).

Compare & Contrast Tables

Commit vs. New baseline vs. New release

	Commit	New baseline	New release
Audience	the team	the project	the users
Action	return updated code to the configuration management repository, resolving conflicts	integrate and thoroughly test the whole; certify the agreed version	deliver baseline code to users
Actor	the programmer	specialized testing team	programmers + business (release frequency is a technical <i>and</i> business decision)
Guarantee	change is shared, conflicts resolved	as bug-free as possible; progress, not regression	value in users' hands
Cadence driver	per change	program size and required quality	technical + business decision; versioned lifespan
Source	Conclusion p.2	Conclusion p.2–4	Conclusion p.9–10

Bugs in baseline vs. Broken baseline

	Bugs in baseline	Broken baseline
Severity	minor	more serious

	Bugs in baseline	Broken baseline
Testing team's verdict	still certify the updated code as the new baseline	reject the buggy commits; sometimes reject the whole work
Fate of the fixes	added to the stack of bug reports, fixed as part of future changes	— (no certification to defer from)
Fate of the increment	proceeds	possibly no new baseline; all work invalidated or postponed — significant cost on large projects
Personal consequence	none stated	committers of buggy files identified; reputation suffers
Source	Conclusion p.6	Conclusion p.7

Large release vs. Small release

	Large release	Small release
Frequency	less frequent	more frequent
Delivery	downloaded and installed	delivered as a patch
Mechanism at the user	replaces the application	incorporated into the user's program by the tool "merge"
Version effect (AwesomeApp 4.2 model)	major number	minor number
Source	Conclusion p.9–10	Conclusion p.9–10

Baseline testing vs. Acceptance testing

	Baseline testing	Acceptance testing
Performed by	specialized testing team (internal)	stakeholders (external)
Question answered	does the integrated whole still work — progress, not regression?	is it what we asked for?
Schedule	at each baseline; overnight/weekend	before release approval
Output	certified (or rejected) baseline	information about progress + approval for release
Drawlets machinery	(not shown for Drawlets)	record/playback acceptance tests; functional tests via JGiven + Mockito
Source	Conclusion p.3–7	Conclusion p.8; Drawlets p.27

Wrong way #1 vs. Wrong way #2 (concept location)

	First wrong turn	Second wrong turn
From	<code>SimpleApplet</code>	<code>DrawingCanvas</code> interface
Into	the palette/UI branch (<code>ToolBar</code> / <code>ToolPalette</code>)	the container implementation <code>SimpleDrawingCanvas</code>
Hypothesis	figure properties are managed by the visible UI components	figure properties are stored by the canvas that holds figures

	First wrong turn	Second wrong turn
Why wrong	palettes hold styles and tool buttons, not per-figure state	a container holds figures but not a figure's own properties
Greyed at backtrack	<code>ToolPalette</code>	<code>SimpleDrawingCanvas</code>
Corrective move	follow the canvas dependency (blue arrow to <code>DrawingCanvas</code>)	go <i>up</i> the interface chain to <code>Figure / AbstractFigure</code>
Source	Drawlets p.10–11	Drawlets p.13–14

Move the code vs. Split the roles (the two refactorings)

	Moving the code	Splitting of roles
Smell addressed	the same logic duplicated across every figure-creating class (<code>basicNewFigure(...)</code> 's "if created at wrong location, move it" part)	one method serving two different roles with the same code
Mechanics	extract new method <code>moveFigure(...)</code> ; make it a member of base class <code>ConstructionTool</code>	split <code>move(...)</code> in <code>AbstractFigure</code> into <code>move(...)</code> and <code>secureMove(...)</code>
Where the owner check then lives	the one consolidated location (plus the four leaf call sites — yellow + cyan on p.34)	only in <code>secureMove(...)</code> (user-requested moves); creation-time <code>move(...)</code> untouched
Classes modified for the change	8	5
LOC modified	95	87
Stated minuses	—	new unit-test set must be created (test code duplicated); new tests cost more effort than adapting existing ones
Source	Drawlets p.33–34, p.37	Drawlets p.35–38

Concept categories at a glance

Cate gory	Meaning	Drawlets members	What you do with them
Signif icant	already in the system's design, central to the change	figure, canvas	anchor the code search on them
Exter nal	cross the system boundary; not yet in the code	owner, ID, password	attach them at the located class (they become <code>OwnerIdentity</code> etc.)
Irrele vant	request-language noise	implement, user, allowed, modify, beginning, input, created (and the unmarked <code>session</code>)	discard before searching
Sourc e	Drawlets p.8	Drawlets p.8	Drawlets p.8, p.16

Slide-by-Slide Companion — Conclusion Deck

A per-page digest of all 10 pages, so every slide is retrievable on its own. Quotes are the slide's own wording.

Conclusion p.1 — title and phase diagram

Title: "Conclusion of software change." The slide places Conclusion at the bottom of the phase stack Initiation → Concept Location → Impact Analysis → Prefactoring → Actualization → Postfactoring → Conclusion, with VERIFICATION as a vertical band beside all of them. Two bullets: it is "the last phase of software change," and "the activities depend on the specific software process." Everything else in the deck elaborates those two sentences.

Conclusion p.2 — steps of conclusion

Three steps named: **Commit** — "programmers return their updated code to the configuration management repository," "resolving conflicts"; **New baseline**; **New release**. Note the asymmetry: only Commit gets sub-bullets here; baseline and release each get their own later slides (p.3–7 and p.9–10).

Conclusion p.3 — new baseline

"A thorough testing is required": it "will guarantee that the new baseline is as bug free as possible" and that "the new baseline represents a progress of the project, not a regression." Cost side: "the baseline testing can take a significant time," is "often done overnight or over the weekend," and "a specialized testing team conducts testing."

Conclusion p.4 — baseline testing (frequency)

"The frequency depends on the size of the program and required quality." Both failure modes are stated: postponing too long "means that there could be a large accumulation of the bugs" which "will make the task of the testing difficult"; while "too frequent baseline testing can represent an unnecessary overhead."

Conclusion p.5 — baseline as a deadline

"Deadline to commit" = "time when the testing of the baseline starts." "Miss the deadline" → "submit by the next deadline," with "additional work needed for the new baseline," which "can be a significant additional work"; and "management knows how often a particular programmer missed the deadline" and "may require an explanation."

Conclusion p.6 — bugs in baseline

"Programmers commit a faulty code — bugs are discovered during the baseline testing." "If the bugs are minor, the testing team can still certify the updated code as the new baseline," "add correction of these bugs into the stack of the bug reports," and "they will be fixed as a part of the future changes."

Conclusion p.7 — broken baseline

"More serious bugs": "the testing team can reject the buggy commits." "Sometimes the whole work done on the new baseline has to be rejected — no new baseline is created," "all work that was done is invalidated or postponed," "a significant cost on large projects." Accountability: the "testing team can identify the programmers who committed buggy files" and the "reputation of these programmers suffers."

Conclusion p.8 — stakeholder's role

Stakeholders "conduct acceptance testing," defined on-slide as "a functional testing done by the stakeholders" that "thoroughly tests software functionalities." Purposes: "it gives them information about progress of the project," and "stakeholders approve software for the release."

Conclusion p.9 — new release

"From time to time, the programmers release the baseline code to the users." "A substantial extra work is required," and "the frequency of the releases is both a technical and a business decision." Cadence pattern: "less frequent large releases + more frequent small releases," illustrated by "application AwesomeApp 4.2" and named the "versioned model of software lifespan."

Conclusion p.10 — large and small releases

Delivery mechanics: "download and install large releases"; "small releases are delivered as patches," "incorporated into the users program by the tool 'merge'."

Slide-by-Slide Companion — Drawlets Deck

A per-page digest of all 38 pages. Every slide carries the footer "© 2012 Václav Rajlich, *Software Engineering: The Current Practice*, Ch. 17" — the Rajlich chapter cross-reference for the whole worked example [Raj13]. Diagram slides are described from the verified renders.

Drawlets p.1 — example of software change (title)

Introduces the **application framework Drawlets**: it "adds graphical display to a host application" and provides a **drawing canvas** with "lines, free-hand lines, rectangles, rounded rectangles, triangles, pentagons, polygons, ellipses, text boxes, images" — ten figure kinds, worth listing exhaustively.

Drawlets p.2 — Drawlets features

"More than 100 classes, 35 interfaces and 40,000 lines of code"; "originally implemented by Kent Beck, Ward Cunningham"; "later ported into Java"; marketed as a "perfect API" with the source URL rolemodelsoft.com/aboutUs/drawlets.htm.

Drawlets p.3 — the host application

The host is "responsible for providing an instance of the drawing canvas, toolbars, and tool buttons." "Class SimpleApplet is a host application," "is a part of the Drawlet library," and "SimpleApplet runs in any browser."

Drawlets p.4 — SimpleApplet window (screenshot)

The running UI: icon toolbar with the **ID** button (top), tool palette (left), canvas displaying one of each figure family (centre), and the style palette with colour swatches, **NO**, and **Fill / Line / Text / Back / Fore / Apply** buttons (right). See "The SimpleApplet window in detail" above for the full reading.

Drawlets p.5 — top classes (diagram)

The full class graph in lollipop notation with the comprehension starting set — `StylePalette`, `SimpleApplet`, `ToolBar`, `ToolPalette` — filled green. See "The Drawlets Class Graph — Complete Structural Reference" for

the node-by-node breakdown.

Drawlets p.6 — change request

Verbatim: "Implement an owner for each figure. An owner is the user who put the figure onto the canvas, and only the owner should be allowed to modify it. At the beginning of a session, the users input their ID and password and they are the owners of all figures that were created during the session." Justification: "this change will make SimpleApplet more versatile and useful — support for cooperative work."

Drawlets p.7 — concepts

The same request text re-shown for concept extraction; the candidate concepts are the request's substantive words (implement, owner, figure, user, canvas, allowed, modify, beginning, session, input, ID, password, created) which the next slide classifies.

Drawlets p.8 — concept classification (table)

The 13-row × 3-column table (irrelevant / external / significant). Significant: **figure, canvas** (bold). External: **owner, ID, password**. Irrelevant: implement, user, allowed, modify, beginning, input, created; the `session` row is unmarked. Full table reproduced in Concept 11.

Drawlets p.9 — locating figure properties

The search question is posed on the class graph: only `SimpleApplet` is green (the comprehended start). All other nodes white: nothing else is yet known to the search.

Drawlets p.10 — wrong way (first)

`SimpleApplet` orange (under examination); `StylePalette`, `ToolBar`, `ToolPalette` green; thick violet arrow down from `SimpleApplet` into the palette branch — the first incorrect hypothesis (figure properties in the visible UI components).

Drawlets p.11 — backtrack (first)

`ToolPalette` flips grey: the palette branch is abandoned. The search stands again at orange `SimpleApplet`.

Drawlets p.12 — right way (first)

Thick blue arrow from `SimpleApplet` to the `DrawingCanvas` interface (green circle): following the canvas dependency is the productive direction.

Drawlets p.13 — wrong way (second)

`DrawingCanvas` circle orange (now examined); thick violet arrow down onto `SimpleDrawingCanvas` (green while inspected); `SequenceOfFigures` circle green above. Second incorrect hypothesis: the canvas implementation stores figure properties.

Drawlets p.14 — backtrack (second)

`SimpleDrawingCanvas` flips grey: the container branch is abandoned; the search returns to the orange `DrawingCanvas` interface.

Drawlets p.15 — summary

The successful path drawn end-to-end with thick blue arrows through orange waypoints: `SimpleApplet` → `DrawingCanvas` → `SequenceOfFigures` → `Figure` → **`AbstractFigure` in red** — the located concept. Grey scars: `ToolPalette`, `SimpleDrawingCanvas`.

Drawlets p.16 — actualization

Two classes red (`AbstractFigure`, `SimpleDrawingCanvas`), two new classes in light blue (`OwnerIdentity`, `SimpleListener`), thick blue arrow `OwnerIdentity` → `AbstractFigure`, thick orange arrow into `SimpleDrawingCanvas`. The change exists; propagation has not begun.

Drawlets p.17 — change propagation

The changed classes throw inconsistency onto their interfaces: blue arrow `AbstractFigure` → `Figure`, orange arrow `SimpleDrawingCanvas` → `DrawingCanvas`; both interface circles green (marked).

Drawlets p.18 — propagation – 1

`DrawingCanvas` orange (worked); orange arrows fan to `SimpleApplet`, `CanvasTool` (both green/marked) and up to `SequenceOfFigures` (green).

Drawlets p.19 — propagation – 2

`SimpleApplet` resolves **red** (changed — session/ID owner); orange arrows fan from it to `StylePalette`, `ToolBar`, `ToolPalette` (all green/marked); `CanvasTool` and `SequenceOfFigures` resolve grey (unchanged).

Drawlets p.20 — propagation – 3

The three palettes resolve grey. Red so far: `AbstractFigure`, `SimpleDrawingCanvas`, `SimpleApplet`. The worklist is briefly nearly empty.

Drawlets p.21 — propagation – 4

Figure interface turns red (the deck's single interface modification); blue arrows fan from it marking `LocatorConnectionHandle`, `Locator`, `StylePalette` (re-marked), `SelectionTool`, `ConstructionTool`, `LabelTool`, `SequenceOfFigures` green. `AbstractFigure` shown pink (changed, settled).

Drawlets p.22 — propagation – 5

Marks resolve to changes: `LocatorConnectionHandle`, `StylePalette`, `SelectionTool`, `ConstructionTool`, `PrototypeConstructionTool` now red (with `AbstractFigure`, `SimpleApplet`, `SimpleDrawingCanvas`, `Figure`). Blue arrows mark `ShapeTool` and `LabelTool` (green); `Locator` / `SequenceOfFigures` grey.

Drawlets p.23 — propagation – 6

`LabelTool` resolves red; `ShapeTool` orange (frontier); blue arrows run along the bottom row marking all four leaf creation tools green.

Drawlets p.24 — propagation – done

Final state: 13 red classes + red `Figure` circle + 2 blue added classes; grey `ToolBar`, `ToolPalette`, `CanvasTool`, `Locator`, `SequenceOfFigures`; orange `ShapeTool` and `DrawingCanvas`. No green marks remain

— the worklist is empty. Full reconciliation with p.37 in the Worked Example.

Drawlets p.25 — unit tests

The suite: "385 unit tests," "1369 assertions," "4800 lines."

Drawlets p.26 — functional tests

Scenario catalogue: "Draw — figure drawn on the canvas, for all figures"; "Select — figure is selected"; "Move — figure is moved to a different location"; the slide elides the rest with an ellipsis and totals "141 functional test cases."

Drawlets p.27 — creation of acceptance tests

"Record/playback tool for Java applications" used to create acceptance tests; "tool JGiven and Mockito used to run the functional tests."

Drawlets p.28 — phase of actualization (tests)

Three test workstreams during actualization: "new unit tests for new classes"; "new functional tests for the new functionality"; and "during the change propagation, the old tests that were impacted by the change were updated."

Drawlets p.29 — test suite maintenance

Three rules: tests from the old version "not affected by the change are kept as regression tests for the future"; "obsolete tests are removed"; "tests of the new features are added after the change."

Drawlets p.30 — results

Baseline: "production code 17800 lines," "unit test code 4800 lines." Change: "91 lines of production code (0.5%) were modified," "124 lines (2.5%) of the test code," "approximately 1.4 test code lines per modified production code line."

Drawlets p.31 — refactoring

"Long change propagation is a problem — it would be advantageous to shorten it." "Refactoring can shorten the change propagation": "move code affected by change into fewer classes"; "split the roles."

Drawlets p.32 — propagation (clean reference)

The p.24 end-state redrawn with transient marks cleared: 13 red classes, red `Figure`, blue `OwnerIdentity / SimpleListener`, everything else white. This is the "no refactoring" picture the next slides improve on.

Drawlets p.33 — moving the code

"Each class that creates new figures has the function `basicNewFigure(...)`" with "two main parts: create a new figure; if the figure was created at wrong location, move it." "We refactored a new method called `moveFigure(...)`" and "made it a member of the base class `ConstructionTool`."

Drawlets p.34 — refactoring impact

All-white graph except `ConstructionTool` (yellow — new home of `moveFigure`) and the four leaf creation tools (cyan — consolidated call sites): the owner-relevant code now occupies a five-class pocket instead of the whole upper graph.

Drawlets p.35 — splitting of roles (principle)

"Same method used in two different roles — the same code can do both jobs." "Propagating change highlights the differences": "only one of the roles needs to be updated," "the other one can stay unchanged," and "splitting the two roles and updating only one of them shortens the propagation."

Drawlets p.36 — splitting the roles (applied)

"Function `move(...)` in `AbstractFigure` is used in two ways": "to move the figure as requested by the user — must check user identity"; "as a part of the new figure creation — does not need to check user identity." "We split `move(...)` and `secureMove(...)`."

Drawlets p.37 — numerical data

The comparison table: classes added 2 / 2 / 2; interfaces modified 1 / 1 / 1; classes modified 13 / 8 / 5; LOC modified 91 / 95 / 87 — for no refactoring / move function / splitting.

Drawlets p.38 — conclusions

"Splitting up a method into two similar methods may require least work." "Minus: a new set of unit tests must be created, thus test code is duplicated." "Minus: more effort is required to create new tests compared to the effort required to adapt existing tests to their changed implementation."

Cross-Lecture Connections

- **The Conclusion deck's p.1 phase diagram is the course's master diagram.** Initiation (L4), Concept Location (L5), Impact Analysis (L6), Prefactoring/Refactoring (L7), Actualization (L8), Postfactoring, Verification (L9) — Lecture 10 supplies the missing last phase (Conclusion p.1) and then replays *all* of them in miniature on Drawlets. If you can narrate the Drawlets change, you have a worked example for every earlier lecture's exam question.
- **Concept location (L5) is enacted, not just defined.** The extract-classify-search pipeline (Drawlets p.7–8) and the grep-like graph navigation with backtracking (p.9–15) are the L5 methodology with real class names attached — including the lesson that the search is fallible by design.
- **Change propagation and actualization (L8) get their canonical animation.** The marked-worklist reading of p.16–24 (green = marked, orange = in hand, red = changed, grey = cleared) is the operational definition of inconsistency-driven propagation, and the `Figure`-interface fan-out (p.21) is the strongest available illustration of why interface edits dominate impact.
- **Refactoring (L7) is justified economically.** "Long change propagation is a problem... refactoring can shorten the change propagation" (Drawlets p.31) plus the $13 \rightarrow 8 \rightarrow 5$ table (p.37) turn refactoring from a code-aesthetics topic into a cost-control instrument — with p.38's test-duplication caveat as the counterweight.
- **Verification (L9) closes the loop with real numbers.** Unit/functional/acceptance layering (p.25–27), the three actualization-time test workstreams (p.28), and the regression-keep/obsolete-remove/new-

add maintenance rules (p.29) instantiate the L9 testing vocabulary; baseline testing and stakeholder acceptance (Conclusion p.3–8) are its team-scale counterparts.

- **The versioned model of software lifespan (Conclusion p.9) links forward** to the course's software-lifespan picture: each release is a visible tick of the staged lifespan, with patches-via-merge (p.10) as its fine-grained mechanism.

Source Map

Source	Page(s)	Content
Conclusion	p.1	Conclusion is the last phase; activities depend on the software process; phase diagram (Verification band).
Conclusion	p.2	Three steps: Commit (return code to CM repo, resolve conflicts), New baseline, New release.
Conclusion	p.3	Baseline testing: thorough (progress-not-regression), expensive (overnight/weekend, specialised team).
Conclusion	p.4	Baseline frequency trade-off: postpone → bug accumulation; too frequent → overhead.
Conclusion	p.5	Baseline as deadline; missing it → re-submit next baseline, extra work, management visibility.
Conclusion	p.6	Bugs in baseline (minor): still certify; add to bug-report stack; fix in future changes.
Conclusion	p.7	Broken baseline (serious): reject commits; whole increment may be invalidated; reputation suffers.
Conclusion	p.8	Stakeholder role: acceptance testing (functional), gives progress info, approves release.
Conclusion	p.9	New release: extra work; technical+business decision; large + small releases; versioned lifespan.
Conclusion	p.10	Large releases downloaded/installed; small releases = patches merged via "merge."
Drawlets	p.1	Drawlets = framework adding a drawing canvas to a host app; figure types listed; Rajlich Ch. 17.
Drawlets	p.2	Features: >100 classes, 35 interfaces, ~40 000 LOC; Beck & Cunningham; ported to Java; "perfect API."
Drawlets	p.3	Host application; SimpleApplet is the bundled host; runs in any browser.
Drawlets	p.4	SimpleApplet window (palette, canvas with shapes, style controls, ID button).
Drawlets	p.5	Top classes diagram; host-reachable classes (StylePalette , SimpleApplet , ToolBar , ToolPalette) highlighted.
Drawlets	p.6	Change request : owner per figure; only owner may modify; ID/password at session start; rationale (cooperative work).
Drawlets	p.7	Concepts extracted from the request.
Drawlets	p.8	Concept classification table: figure/canvas significant; owner/ID/password external; rest irrelevant.
Drawlets	p.9	Concept location starts at SimpleApplet (locating figure properties).
Drawlets	p.10, 13	"Wrong way": candidate SimpleDrawingCanvas examined and rejected.

Source	Page(s)	Content
Drawlets	p.11, 14	"Backtrack": abandon dead-end, classes greyed out, return to last good class.
Drawlets	p.12	"Right way": follow to the <code>Figure / AbstractFigure</code> abstraction.
Drawlets	p.15	"Summary": <code>AbstractFigure</code> (red) located as the class holding figure properties → owner goes here.
Drawlets	p.16	Actualization: new classes <code>OwnerIdentity</code> + <code>SimpleListener</code> (blue); change begins at <code>AbstractFigure</code> (red).
Drawlets	p.17	Change propagation begins; inconsistency spreads from <code>AbstractFigure / Figure</code> outward.
Drawlets	p.18–23	Propagation 1–6: frontier visits <code>SimpleApplet</code> , palettes, canvas, tool tree, down to leaf figure tools.
Drawlets	p.24	"Propagation – done": large red set of modified classes; grey checked-unchanged; 2 new classes.
Drawlets	p.25	Unit tests: 385 tests, 1369 assertions, 4800 LOC.
Drawlets	p.26	Functional tests: Draw/Select/Move/...; 141 functional test cases.
Drawlets	p.27	Acceptance tests via record/playback tool; functional tests run with JGiven + Mockito.
Drawlets	p.28	Actualization's test work: new unit tests, new functional tests, update impacted old tests.
Drawlets	p.29	Test-suite maintenance: keep unaffected as regression, remove obsolete, add new-feature tests.
Drawlets	p.30	Results: 17 800 production LOC + 4800 test LOC; change = 91 LOC (0.5 %) prod + 124 LOC (2.5 %) test; ~1.4 test/prod.
Drawlets	p.31	Refactoring shortens propagation: move code into fewer classes; split roles.
Drawlets	p.32	Propagation diagram (pre-refactor reference state, mostly red).
Drawlets	p.33	Move the code: extract <code>moveFigure(...)</code> from <code>basicNewFigure(...)</code> , pull up into <code>ConstructionTool</code> .
Drawlets	p.34	Refactoring impact: smaller impacted set (<code>ConstructionTool</code> + four creation tools).
Drawlets	p.35	Splitting roles: same method, two roles; only one needs updating; shortens propagation.
Drawlets	p.36	Split <code>AbstractFigure.move(...)</code> into <code>move(...)</code> and <code>secureMove(...)</code> (identity check only in secure).
Drawlets	p.37	Numerical comparison: classes modified 13 / 8 / 5; LOC 91 / 95 / 87; +2 classes, 1 interface each.
Drawlets	p.38	Conclusions: splitting may be least work, but duplicates test code and costs more new-test effort.